Joint AI Service Placement, Task Scheduling, and Resource Allocation for IoT in 6G Networks

Zhenyu Zhang, Lu Lu, Qin Li, Yuhao Chai, Di Wu, and Yong Zhang, *Member, IEEE*

***Abstract*—As Internet of Things (IoT)-based artificial intelligence (AI) applications grow, the surge in computational and communication demands has raised concerns about energy consumption, making it critical for 6G networks to address this challenge. This article examines the joint optimization of AI service placement, task scheduling, and computing resource allocation in an edge-network-cloud system to minimize long-term energy consumption. These problems are interdependent: AI service placement determines service locations, influencing task scheduling, which in turn dictates computing resource allocation. The key challenge lies in the coupling of these variables and the two time-scale nature of the problem, involving long-term (AI service placement) and short-term (task scheduling and computing resource allocation) strategies. To address this, a hierarchical Markov decision process (HMDP) framework is proposed for efficient and coordinated optimization across time scales. A hierarchical mean-field dueling double deep Q-network (HMFD3QN) algorithm is developed within this framework, where the upper layer optimizes AI service placement, and the lower layer manages task scheduling and computing resource allocation. By integrating mean-field theory, the algorithm reduces the complexity of multiagent interactions. The computing resource allocation problem is shown to be convex when other variables are fixed, and an optimal strategy is derived using Karush–Kuhn–Tucker (KKT) conditions to simplify the action space for reinforcement learning. Experimental results demonstrate that the proposed method can reduce energy consumption by up to 34% compared to baseline methods, significantly improve queue stability, and increase the proportion of tasks meeting QoS requirements.**

***Index Terms*—Artificial intelligence (AI) service placement, digital twin, Internet of Things (IoT), mean-field multiagent reinforcement learning (MARL), task scheduling.**

I. Introduction

With the rapid advancement of Internet of Things (IoT)-based artificial intelligence (AI) applications, such as industrial Internet, smart grids, autonomous driving, and telemedicine, network transmission, and computing capabilities face significant challenges [1], [2], [3], leading to substantial energy consumption. Due to the limited computing and storage capacity of IoT terminals, current 5G networks primarily rely on an "edge-cloud" architecture to deliver AI services. In contrast, 6G networks deeply integrate computing and networking resources, constructing an "edge-network-cloud (E-N-C)" architecture [4]. This enables collaborative operation of computing resources across the edge, network, and cloud, allowing for on-demand allocation and flexible scheduling of resources. Unlike 5G networks, which mainly support traditional services, 6G networks seamlessly host AI services. Supported by the E-N-C architecture, 6G establishes AI-native infrastructure, enabling efficient deployment and optimization of AI applications [5]. By embedding AI models and computing capacity into network resources, 6G networks adopt a task-centric approach for unified and flexible orchestration.

Leveraging this architecture, 6G networks facilitate the construction of a joint management mechanism for AI services, optimizing processes from service placement to task scheduling and computing resource allocation in an integrated manner. However, with the increasing scale and complexity of AI models, energy consumption has become a critical challenge, particularly in the context of the "carbon neutrality" era and growing global energy concerns. The energy demands of AI inference, spanning edge, network, and cloud scenarios, require urgent attention to ensure the sustainable development of 6G systems. Therefore, this article aims to jointly optimize AI service placement, task scheduling, and computing resource allocation to minimize long-term energy consumption within the E-N-C system. To the best of our knowledge, no existing research comprehensively addresses this joint problem.

As illustrated in Fig. 1, this article models the processes of AI service placement, task scheduling, and computing resource allocation in the E-N-C system. AI service placement involves downloading relevant model repositories and data to computing nodes via cloud servers, where inference is performed using serverless computing. Serverless computing offers flexible resource scheduling and efficient execution environments [6], [7], [8]. Based on differences in service frequency and resource utilization efficiency, we can divide AI services into two types: 1) continuous and 2) occasional. AI tasks are offloaded to computing nodes where the corresponding services are placed, and specific models are selected

Received 21 March 2025; revised 17 May 2025 and 22 June 2025; accepted 7 July 2025. Date of publication 10 July 2025; date of current version 8 September 2025. This work was supported by the Beijing University of Posts and Telecommunications-China Mobile Research Institute Joint Innovation Center. *(Corresponding author: Yong Zhang.)*

Zhenyu Zhang, Yuhao Chai, and Di Wu are with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zhangzhenyucad@bupt.edu.cn; chaiyh@bupt.edu.cn; diwubupt@163.com).

Lu Lu and Qin Li are with the Department of Basic Network Technology, China Mobile Research Institute, Beijing 100053, China (e-mail: lulu@chinamobile.com; liqinyjy@chinamobile.com).

Yong Zhang is with the Beijing Key Laboratory of Work Safety Intelligent Monitoring, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: yongzhang@bupt.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3587979

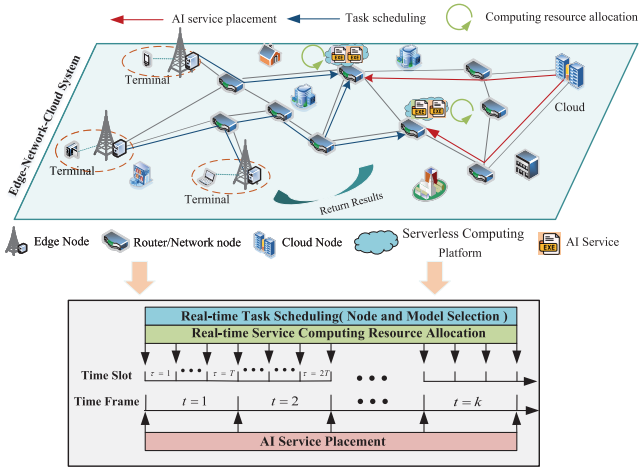


Fig. 1. E-N-C system with strategy decisions at two timescales.

for inference based on task requirements. Computing nodes dynamically allocate resources for each service to improve resource utilization efficiency.

Addressing this joint optimization problem is challenging as it involves optimization across two time scales. Task scheduling (including node and model selection) and computing resource allocation are made at each time slot, representing a short-term time scale, while service placement decisions are made at longer intervals, corresponding to a long-term time scale, as shown in Fig. 1. These decisions are interdependent, making it difficult to decouple optimization variables across time scales. Furthermore, service placement and task scheduling decisions are discrete, while resource allocation decisions are continuous, resulting in a two time-scale mixed-integer nonlinear programming (MINLP) problem. While existing multitime-scale frameworks in mobile-edge computing (MEC) have proven effective for resource allocation and task offloading in traditional telecommunication services [9], [10], [11], they are inadequate for AI tasks. Specifically, these frameworks fail to address the unique requirements of AI services, such as inference accuracy, and do not consider AI-specific resource allocation challenges, including AI service placement and model selection. Moreover, unlike MEC's edge-cloud architecture, this article considers the distribution of computing resources across the network, further complicating the problem.

To address these challenges, multiagent reinforcement learning (MARL) [10], [12], [13] emerges as a promising approach due to its capacity to learn in dynamic and complex systems. However, optimizing AI service placement, task scheduling and resource allocation involves two types of heterogeneous agents: 1) Computing nodes, responsible for long-term global optimization. 2) Terminals, handling short-term task requests with finer decision-making granularity. This heterogeneity, coupled with differences in temporal granularity, significantly increases system complexity. Additionally, as the number of computing nodes and terminals grows, MARL faces scalability challenges, such as poor convergence and high communication overhead, necessitating the development of more efficient and scalable solutions.

To address the two-time-scale multi-AI service placement and task scheduling problem, this article proposes a hierarchical mean-field dueling double deep Q-network (HMFD3QN) algorithm. The computing resource allocation problem is decoupled from the overall objective and proven to be a convex function. Accordingly, a dynamic computing resource allocation algorithm based on the Karush–Kuhn–Tucker (KKT) conditions is developed. The main contributions of this article are summarized as follows.

1) A two-time-scale communication, queuing, and computing framework is established to jointly optimize AI service placement, task scheduling, and resource allocation. It also considers the varying computing capacities of nodes, diverse types of AI services, multiple AI models, and differing QoS requirements.
2) To address the temporal granularity disparities between AI service placement (long-term decisions) and task scheduling (short-term decisions), a hierarchical Markov decision process (HMDP) framework is developed, facilitating efficient and coordinated optimization of both long-term and short-term objectives.
3) The HMFD3QN algorithm is proposed building upon the HMDP framework. This algorithm integrates mean-field game theory to effectively reduce communication overhead in large-scale multiagent scenarios, while supporting the layered optimization of service placement and task scheduling.

The remainder of this article is organized as follows. The Section II introduces the related work. Section III introduces the system model and problem description. Section IV details the problem-solving process. Section V presents the design of the HMFD3QN algorithm. Section VI evaluates the performance of the algorithm. Finally, Section VII summarizes this article.

II. Related Work

A. AI Service Placement and Task Scheduling

Studies on AI service placement and task scheduling are limited, with most existing studies focusing on model placement and task scheduling. The AI model placement problem focuses on deploying individual models as the smallest unit [1], [14], [15], [16]. For m types of AI services, each with n models, the decision space at each computing node is $m \times n$. Once the models are placed, terminals only need to decide the offloading node hosting the desired model, which involves a single decision variable. In contrast, AI service placement treats an entire AI service (encompassing multiple models) as a unified deployment unit [17], reducing the decision space at each node to m, as the service is deployed as a whole. However, terminals must make two interdependent decisions: first, selecting the offloading node hosting the required service type, and second, choosing the most suitable model version within the service. These interdependent decisions significantly increase the optimization complexity.

Some studies simplify decision-making processes across two different time scales into a time scale. For example, in edge intelligence scenarios, an ADMM-based algorithm

has been proposed to jointly optimize resource allocation and computing offloading [18]. In this method, the cloud server broadcasts AI services to edge computing nodes, while computing offloading requests are processed within the same timescale. Similarly, in vehicular edge computing, a time-period-aware (TPA) algorithm with a guaranteed approximation ratio has been introduced to jointly address AI model placement and task offloading for multitask inference [14].

Some studies apply penalty mechanisms to reduce frequent AI model placements. For example, a DDPG-based algorithm optimizes deep neural network (DNN) deployment, task offloading, and resource allocation by penalizing frequent DNN deployments across consecutive time slots [1]. Similarly, in MEC, AI model caching strategies are generated by incorporating switching costs into the reward function, while task offloading is handled using a greedy strategy [19].

Other studies focus on subproblems like AI task scheduling, assuming that AI services are already deployed. For instance, in endogenous AI wireless networks, an NSG-TSRA heuristic algorithm enables flexible adjustments of data quality and resource allocation [20]. To optimize energy efficiency in D2D system, a two-layer approach uses a heuristic algorithm for offloading decisions in the outer layer and a subgradient-based algorithm for resource allocation in the inner layer [21]. Similarly, studies on AI service placement sometimes assume that terminals are bound to specific computing nodes, eliminating the need for task scheduling decisions. An multiparadigm deployment model (MPDM) has been proposed to balance system accuracy, service scale, and deployment cost [15]. To handle placement under uncertainty, a sequence-to-sequence (S2S) neural network with uncertainty estimation generates placement locations for multiple AI models [16].

In summary, the joint optimization of AI service placement (long-term) and task scheduling (short-term) across two time scales remains a significant challenge. Existing approaches often simplify the time scales or introduce penalty terms to relax constraints. However, simplifying the problem to a single time scale—where decisions on AI service placement and task scheduling must be made in each small time slot (e.g., each second)—dramatically increases the frequency of decision-making. This leads to amplified decision complexity and excessive resource consumption due to continuous adjustments. Moreover, this article incorporates the consideration of computing resource allocation, adding yet another layer of complexity. Consequently, the problem becomes a two timescale MINLP problem, for which existing research has yet to provide an effective solution.

B. Resource Orchestration Based on Mean-Field MARL

Although MARL has demonstrated remarkable adaptability and performance in dynamic and complex decision-making scenarios, it still faces challenges, such as the curse of dimensionality and high computing resource consumption. Mean-field game theory addresses these issues by simplifying complex multiagent interactions into interactions between individual agents and a mean field [22]. This approach not only alleviates the curse of dimensionality but also significantly enhances computing efficiency and scalability.

For instance, in D2D networks, mean-field reinforcement learning (MFRL) replaces complex user interactions with a mean action, enabling efficient resource allocation and maintaining stability as the number of users grows [23]. Similarly, in UAV scenarios, mean-field game theory reduces complexity by modeling collective UAV behavior as a mean-field term, optimizing trajectories, data collection, and minimizing the Age of Information (AoI) [24]. For UAV communication security, mean-field-enhanced distributed learning uses virtual mean agents to reduce communication costs and optimize energy consumption [25].

In wireless sensor networks (WSNs) and mobile WSNs (MWSNs), mean-field approaches simplify the interactions among sensors by modeling their collective effects as mean fields, enabling decentralized power control, energy-efficient routing, and solving coupled HJB-FPK equations for distributed optimization [26], [27]. In vehicular communication, mean-field methods approximate neighboring agents' influence as a mean action using the FPK equation, optimizing spectrum and power allocation while reducing interference and enhancing connectivity [28]. These applications demonstrate the effectiveness of mean-field theory in managing large-scale, complex systems.

However, mean-field-based multiagent methods are primarily designed to solve static solutions in stochastic optimization problems for large-scale agent scenarios, making them unsuitable for direct application to two-time-scale optimization problems with interdependent constraints. Furthermore, in our scenario, computing nodes and terminals represent two heterogeneous types of agents, each operating on different time scales and responsible for distinct decision-making tasks. As a result, they cannot be treated as a single type of agent. This necessitates the development of a customized MARL algorithm capable of optimizing decision-making across different time scales.

In summary, this article proposes a joint optimization problem for AI service placement, task scheduling, and computing resource allocation, formulated as a two-timescale MINLP problem for which no effective solution currently exists. While existing studies typically simplify the problem through timescale merging or constraint relaxation, these methods substantially escalate decision complexity and resource overhead. To address these limitations, we develop a hierarchical mean-field MARL algorithm that integrates the dynamic decision-making capabilities of MARL with mean-field game theory. The proposed solution addresses the limitations of current studies that focus on static and homogeneous agents, enabling effective collaborative optimization across heterogeneous agents (e.g., computing nodes and terminals) operating at different timescales.

III. System Model and Problem Formulatuion

A. System Model

As shown in Fig. 2, in the E-N-C system, terminals send AI inference task requests to the network via associated

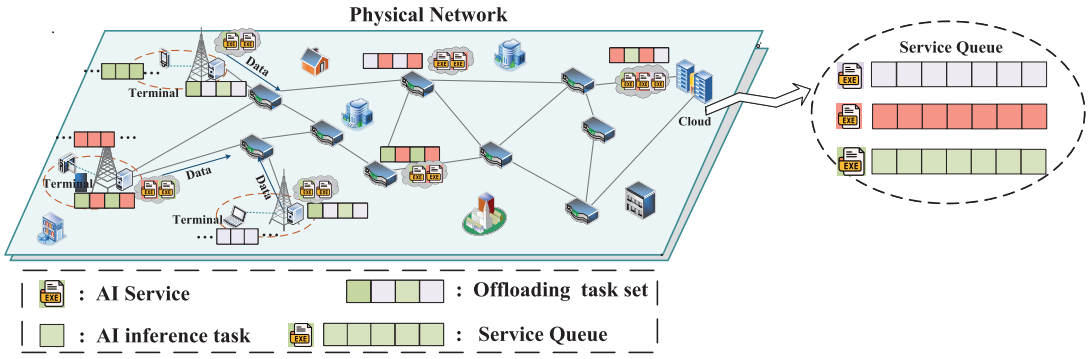


Fig. 2. E-N-C system.

base stations. The system dynamically offloads these tasks to computing nodes where the corresponding types of AI services have been deployed.

The E-N-C system is assumed to operate on a discrete timeline, where the timeline is divided into $K \in \mathbb{N}$ large temporal units, referred to as time frames. Each time frame is further subdivided into $T \in \mathbb{N}^+$ smaller temporal units, called time slots. Given a sequence of time slots $\tau \in \mathcal{T} = \{0, 1, \ldots, KT-1\}$, $t = kT$ (where $k = 0, 1, \ldots$) is defined as the starting point of each time frame $[t, t+T-1]$. The duration of each time slot τ is denoted by ρ.

The system consists of a set of computing nodes, denoted by $\mathcal{V} = \{V_{ed}, V_{\text{net}}, V_{\text{cld}}\}$, where V_{ed}, V_{net}, and V_{cld} represent the sets of edge nodes, network nodes, and cloud nodes, respectively. The transmission rate of the link between nodes v_e and v_j at time slot τ is represented by $R_{e,j}(\tau)$. Each computing node $v \in \mathcal{V}$ is characterized by the following attributes:

$$\left(\{Q_{v,s}(\tau)\}_{s=1}^{|\mathcal{S}|}, C_v(t), f_v(\tau)\right) \quad \forall v \in \mathcal{V} \tag{1}$$

where $\mathcal{S}$ represents the set of AI inference service types, and $|\mathcal{S}|$ denotes the number of services in the set. $\{Q_{v,s}(\tau)\}_{s=1}^{|\mathcal{S}|}$ represents the set of service queues on node v at time slot τ. $C_v(\tau)$ represents the storage capacity of node v at time frame t. Since $C_v(\tau)$ is only used to constrain service placement variables at time frame t, we assume that the storage capacity remains constant within each time frame. $f_v(\tau)$ represents the computing capacity of node v at time slot τ. Specifically, $C_v(t)$ includes both $C_v^{\text{VRAM}}(t)$, which represents the available GPU video random access memory (VRAM), and $C_v^{\text{HDD}}(t)$, which represents the available storage capacity of the hard disk drive (HDD).

In addition to the computing nodes, the system also includes a set of terminals $i \in \mathcal{I}$, which generate AI task requests $P_{i,s}(\tau)$. Specifically, the attributes of each task request can be expressed as

$$P_{i,s}(\tau) = \left(\omega_s, d_{i,s}(\tau), t_{i,s}^{\text{th}}, \text{Acc}_{i,s}^{\text{th}}\right) \tag{2}$$

where the binary variable $\omega_s \in \{0, 1\}$ indicates the type of service requested. The task data size for service s requested by terminal i at time slot τ is denoted as $d_{i,s}(\tau)$, measured in bits. Additionally, $t_{i,s}^{\text{th}}$ and $\text{Acc}_{i,s}^{\text{th}}$ represent the service-level agreement (SLA) deadline and the minimum required inference accuracy, respectively. It is assumed that each terminal can request only one type of service per time slot. For detailed system parameters and symbol representations, please refer to Table I.

B. AI Service Placement

For each service s (i.e., $s \in \mathcal{S}$), inference can be performed using a corresponding set of AI models $\mathcal{M}$. Cloud servers maintain model repositories and databases for all service types. Service placement involves downloading the relevant model libraries and databases from cloud servers to computing nodes. Predeploying these resources to computing nodes can reduce latency. The binary decision variable $x_{v,s}(t)$ indicates whether service s is placed on computing node v during time frame t

$$x_{v,s}(t) \in \{0, 1\} \quad \forall v \in \mathcal{V} \quad \forall s \in \mathcal{S}. \tag{3}$$

On the other hand, downloading the relevant model repositories and databases from cloud servers incurs costs, making frequent changes in service placement impractical. Service placement decisions for each computing node v are determined at the beginning of each time frame t and remain fixed for the duration of that time frame.

AI services are categorized into continuous and occasional services. Specifically, some services, such as environmental parameter monitoring and equipment failure prediction, are generated with high frequency and are thus classified as continuous AI services. The AI models corresponding to these services are kept in a warm state within their respective containers. This can be achieved by using scheduled triggers to periodically invoke functions, thereby keeping the containers active and reducing response time.

On the other hand, services like anomaly detection and emergency response, which are less frequently used, are classified as occasional AI services. Due to the infrequency of these services, on-demand initiation can effectively reduce costs. These services remain in a cold start state, consuming no storage until invoked, but may experience cold start delays due to container initialization.

To further optimize resource allocation, occasional AI services can be stored in cheaper hard disk. In contrast, continuous AI services, which are invoked with high frequency, are

TABLE I
SUMMARY OF NOTATIONS

Notion	Definition
t	Time frame.
$\tau, \mathcal{T}$	Time slot and the set of time slots.
$\mathcal{V}, \mathcal{I}, \mathcal{S}$	The set of computing nodes, IoT terminals, and AI service, respectively.
$R_{i,j}(\tau)$	The transmission rate of the link between nodes v_i and v_j at time slot τ.
$Q_{v,s}(\tau), f_v(\tau)$	The service queue and computing capacity of node v at time slot τ.
$C_v(t)$	The storage capacity of node v at time frame t.
$C_v^{VRAM}(t)$	The available GPU VRAM of node v at time frame t.
$C_v^{HDD}(t)$	The available HDD storage of node v at time frame t.
c_s^{VRAM}	The GPU VRAM storage capacity for continuous service s.
c_s^{HDD}	The HDD storage capacity for occasional service s.
$P_{i,s}(\tau)$	The set of AI inference service requests s from terminal i at time slot τ.
ω_s	Binary decision variable indicating the type of service s.
$d_{i,s}(\tau)$	The task data size for service s of terminal i at time slot τ.
$t_{i,s}^{th}, Acc_{i,s}^{th}$	The SLA deadline and the minimum required inference accuracy.
$x_{v,s}(t)$	Binary decision variable indicating the placement of service s on node v.
$\mathcal{M}_s$	The set of AI models available for AI inference service s.
$F(m_{s,b}), Acc(m_{s,b})$	The computing capacity required and the inference accuracy achieved by model $m_{s,b}$, respectively.
$\gamma_{i,s,b}(\tau)$	Binary decision variable indicating whether the task from terminal i for service s selects model $m_{s,b}$ at time slot τ.
$\delta_{i,s,v}(\tau)$	The offloading decision for the AI task from terminal i for service type s to computing node v at time slot τ.
$t_{s,v}^{cold}$	The cold start latency for initiating an occasional service task ($\omega_s = 0$) on a serverless computing node v.
$T_{i,s}^{tr}, T_{i,s}^{cp}$	The transmission delay and computation delay for service s requested by terminal i.
$E_{i,s}^{tr}, E_{i,s}^{cp}$	The transmission energy and computation energy consumption for service s requested by terminal i.
ε_i^d	The transmission energy consumption coefficient for terminal i.
$\varepsilon_v^c, \varepsilon_v^{cold}$	The computation and cold-start energy consumption coefficient for node v.
$T_{v,s}^{qu}$	The queuing delay of the queue for service s in computing node v.

stored in GPU VRAM to minimize response time. However, due to the limited storage capacity of edge nodes and network nodes, deploying all services simultaneously is not feasible. Furthermore, the storage space occupied by the services placed on computing node v must not exceed the total storage capacity of its GPU VRAM, $C_v^{\text{VRAM}}(t)$, or its HDD storage capacity, $C_v^{\text{HDD}}(t)$, at any given time. These constraints can be expressed as follows:

$$\sum_{s\in\mathcal{S}} x_{v,s}(t)\omega_s c_s^{\text{VRAM}} \le C_v^{\text{VRAM}}(t) \quad \forall v \in \mathcal{V} \tag{4}$$

$$\sum_{s\in\mathcal{S}} x_{v,s}(t)(1-\omega_s)c_s^{\text{HDD}} \le C_v^{\text{HDD}}(t) \quad \forall v \in \mathcal{V} \tag{5}$$

where the variable $\omega_s \in \{0, 1\}$ indicates the service type, with $\omega_s = 1$ signifying a continuous service, and $\omega_s = 0$ indicating an occasional service. c_s^{VRAM} and c_s^{HDD} represent the GPU VRAM storage capacity for continuous service s and the HDD storage capacity for occasional service s, respectively.

Each AI inference service s corresponds to a set of AI models, denoted as $\mathcal{M}_s = \{m_{s,1}, m_{s,2}, \ldots, m_{s,|\mathcal{B}|}\}$, where $b \in \mathcal{B}$. These models encompass varying levels of fine-grained and lightweight computing capabilities. The computing capacity required by model $m_{s,b}$ is represented by $F(m_{s,b})$ (measured in GFLOPS per batch), while its inference accuracy is denoted by $\text{Acc}(m_{s,b})$. At time slot τ, the variable $\gamma_{i,s,b}(\tau)$ indicates whether the task from terminal i for service s selects model $m_{s,b}$. The selection variable $\gamma_{i,s,b}(\tau)$ must satisfy the following condition:

$$\gamma_{i,s,b}(\tau) = \{0, 1\} \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S} \quad \forall b \in \mathcal{B} \tag{6}$$

$$\sum_{b\in\mathcal{B}} \gamma_{i,s,b}(\tau) = 1 \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S}. \tag{7}$$

C. Task Scheduling

Let $\delta_{i,s,v}(\tau)$ represent the offloading node selection for the AI task from terminal i for service type s at time slot τ. Since each task can only be offloaded to a computing node, this condition can be expressed as

$$\delta_{i,s,v}(\tau) = \{0, 1\} \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S} \quad \forall v \in \mathcal{V} \tag{8}$$

$$\sum_{v\in\mathcal{V}} \delta_{i,s,v}(\tau) = 1 \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S}. \tag{9}$$

The transmission delay of the task from terminal i for service type s at time slot τ to the target offloading node v can be calculated as follows:

$$T_{i,s}^{tr}(\tau) = \sum_{v\in\mathcal{V}} \delta_{i,s,v}(\tau)\frac{h_{src_i,v}d_{i,s}(\tau)}{R_{src_i,v}(\tau)} \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S} \tag{10}$$

where src_i denotes the edge node associated with terminal i. The number of hops needed for the inference task to reach the target node v is $h_{src_i,v}$.

The transmission energy consumption is

$$E_{i,s}^{tr}(\tau) = \sum_{v\in\mathcal{V}} \delta_{i,s,v}(\tau)\frac{\varepsilon_i^d h_{src_i,v}d_{i,s}(\tau)}{R_{src_i,v}(\tau)} \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S} \tag{11}$$

where ε_i^d is the transmission energy consumption coefficient.

The cold-start latency for initiating an occasional service task ($\omega_s = 0$) on a computing node v is denoted as $t_{s,v}^{\text{cold}}$. It is assumed, without loss of generality, that $t_{s,v}^{\text{cold}}(\tau) < \rho$. The computation delay is denoted as

$$T_{i,s}^{cp}(\tau) = \sum_{v\in\mathcal{V}}\sum_{b\in\mathcal{B}}\Bigg[x_{v,s}(t)\delta_{i,s,v}(\tau)\gamma_{i,s,b}(\tau)\frac{\big(d_{i,s}(\tau)/D_s\big)F\big(m_{s,b}\big)}{f_{v,s}(\tau)} + (1-\omega_s)\cdot t_{s,v}^{\text{cold}}(\tau)\Bigg] \quad \forall i \in \mathcal{I} \quad \forall s \in \mathcal{S} \tag{12}$$

where D_s (in bits per batch) represents the amount of data contained in each batch for AI service s, and $d_{i,s}(\tau)/D_s$

represents the number of batches in the request. The variable $f_{v,s}(\tau)$ denotes the computing capacity allocated to service s by computing node v during time slot τ, measured in GFLOPS. The computing capacity allocated to different services must not exceed the total available computing capacity at computing node v during any time slot, which can be expressed as

$$\sum_{s\in\mathcal{S}} x_{v,s}(t) f_{v,s}(\tau) \le f_v(\tau) \quad \forall v \in \mathcal{V}. \tag{13}$$

The computation energy consumption is

$$E^{cp}_{i,s}(\tau) = \sum_{v\in\mathcal{V}}\sum_{b\in\mathcal{B}}\Big[x_{v,s}(t)\delta_{i,s,v}(\tau)\gamma_{i,s,b}(\tau)\varepsilon^c_v\big(d_{i,s}(\tau)/D_s\big) F\big(m_{s,b}\big)\big[f_{v,s}(\tau)\big]^2 + (1-\omega_s)\varepsilon^{\text{cold}}_v t^{\text{cold}}_{s,v}(\tau)\Big] \tag{14}$$

where ε^c_v and $\varepsilon^{\text{cold}}_v$ are the computation and cold-start energy consumption coefficients for node v, respectively.

Thus, the sum system energy consumption at time slot τ can be calculated by

$$E(\tau) = \sum_{i\in\mathcal{I}}\sum_{s\in\mathcal{S}}\big(E^{tr}_{i,s}(\tau) + E^{cp}_{i,s}(\tau)\big). \tag{15}$$

D. Service Queuing Model

Let $Q_{v,s}(\tau)$ denote the service queue backlog (in GFLOPs) for service s on computing node v at time slot τ. The length of the queue reflects the queuing time for tasks. Thus, constraining the queuing time is equivalent to ensuring the long-term stability of the task queue. If the computing demand of tasks exceeds the node's computing capacity, queue accumulation will occur, leading to increased queuing delays. The task queue $Q_{v,s}(\tau)$ is updated as follows:

$$\begin{aligned} Q_{v,s}(\tau+1) &= \max\ \big\{Q_{v,s}(\tau) - W_{v,s}(\tau), 0\big\} + A_{v,s}(\tau) \\ W_{v,s}(\tau) &= x_{v,s}(t) f_{v,s}(\tau) \\ A_{v,s}(\tau) &= \sum_{i\in\mathcal{I}}\sum_{b\in\mathcal{B}} \delta_{i,s,v}(\tau)\gamma_{i,s,b}(\tau)\big(d_{i,s}(\tau)/D_s\big)F\big(m_{s,b}\big) \end{aligned} \tag{16}$$

where $W_{v,s}(\tau)$ is the maximum computing capacity allocated by node v for service s during the time interval $[\tau, \tau+1]$. $A_{v,s}(\tau)$ is the computing capacity required by service s on node v at time slot $\tau+1$.

According to Little's Law, the queue delay is proportional to the queue backlog and inversely proportional to the average task arrival rate. The delay violation probability is defined as the probability that the average queue delay exceeds the delay requirement $T^{qu.\max}_{v,s}$, with the objective of imposing constraints on this probability. The average queuing delay of the service queue is used to approximate the constraint on task queuing delay. $T^{qu,\max}_{v,s}$ is the maximum task queuing delay used to optimize the SLA of task s. Therefore, the delay requirement is denoted as $T^{qu}_{v,s}(\tau)$, with the following constraints:

$$\begin{aligned} &T^{qu}_{v,s}(\tau) \\ &= \frac{\lim_{\tau\to\infty}\frac{1}{\tau}\sum_{u=0}^{\tau-1} A_{v,s}(u)}{\Big(\lim_{\tau\to\infty}\frac{1}{\tau}\sum_{u=0}^{\tau-1} W_{v,s}(u)\Big)\Big(\lim_{\tau\to\infty}\frac{1}{\tau}\sum_{u=0}^{\tau-1}\big(W_{v,s}(u) - A_{v,s}(u)\big)\Big)} \\ &\le T^{qu.\max}_{v,s}. \end{aligned} \tag{17}$$

Thus, the task's SLA at time slot τ must satisfy the following constraints:

$$T^{tr}_{i,s}(\tau) + \sum_{v\in\mathcal{V}} \delta_{i,s,v}(\tau) T^{qu.\max}_{v,s} + T^{cp}_{i,s}(\tau) \le t^{\text{th}}_{i,s}. \tag{18}$$

E. Problem Formulation

To improve network performance, the optimization objective is to minimize long-term energy consumption. The optimization variables include AI service placement $x_{v,s}(t)$, offloading node selection $\delta_{i,s,v}(\tau)$, AI model selection $\gamma_{i,s,b}(\tau)$, and service computing capacity allocation $f_{v,s}(\tau)$. Based on the previous discussion, the problem can be formulated as follows:

$$\begin{aligned} \text{P0:}\quad & \min_{x(t),\delta(\tau),\gamma(\tau),f(\tau)} \lim_{|\mathcal{T}|\to\infty}\frac{1}{|\mathcal{T}|}\sum_{\tau} E(\tau) \\ \text{s.t.}\quad & \text{C0-1: (3)–(18)} \\ & \text{C0-2: } \sum_{b\in\mathcal{B}} \gamma_{i,s,b}(\tau)\text{Acc}\big(m_{s,b}\big) \ge \text{Acc}^{\text{th}}_{i,s} \quad \forall i\in\mathcal{I} \quad \forall s\in\mathcal{S} \\ & \text{C0-3: } \lim_{\tau\to\infty}\frac{\mathbb{E}\{Q_{v,s}(\tau)\}}{\tau} = 0 \quad \forall v\in\mathcal{V} \quad \forall s\in\mathcal{S} \end{aligned} \tag{19}$$

where $x(t) = \{x_{v,s}(t)\}$, $\delta(\tau) = \{\delta_{i,s,v}(\tau)\}$, $\gamma(\tau) = \{\gamma_{i,s,b}(\tau)\}$, $f(\tau) = \{f_{v,s}(\tau)\}$ $\forall i \in \mathcal{I}$ $\forall v \in \mathcal{V}$ $\forall s \in \mathcal{S}$ $\forall b \in \mathcal{B}$. Constraint C0-2 ensures that each inference task meets the minimum accuracy requirements. Constraint C0-3 guarantees the long-term stability of all service queues.

P0 is an MINLP problem, involving three discrete variables, x, δ, γ, and one continuous variable f. Solving P0 directly is extremely challenging due to the problem's scope, which spans all time slots over an infinite horizon, making it impossible to obtain the parameters for these time slots in advance. Moreover, as illustrated in Fig. 1, it involves decision-making across different time scales: task scheduling (e.g., node selection and model selection) and computing resource allocation require real-time decisions at each time slot, while service placement is optimized at the granularity of each time frame, further increasing the complexity of the problem.

IV. Problem Solving

A. Problem Transformation via Lyapunov Optimization

To address this, Lyapunov optimization is introduced to equivalently transform the long-term optimization problem into a short-term optimization problem. Let $Q(t) = [Q_{1,1}(t), Q_{1,2}(t), \ldots, Q_{|\mathcal{V}|,|\mathcal{S}|}(t)]$. The quadratic Lyapunov function is defined to measure queue congestion at time frame $t = kT$. This function is expressed as

$$L(Q(t)) = \frac{1}{2}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\big(Q_{v,s}(t)\big)^2. \tag{20}$$

The T-slot Lyapunov drift is then defined as follows:

$$\Delta_T(Q(t)) = \mathbb{E}\{L(Q(t+T)) - L(Q(t))|Q(t)\}. \tag{21}$$

Based on Lyapunov optimization theory [29], we define the drift-plus-penalty function for time frame t as $\Delta_T(Q(t)) + V\mathbb{E}[\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)]$, where $V \ge 0$ is a parameter that balances the minimization of total energy consumption and Lyapunov drift. Service placement decisions $x(t)$ are made at

each time frame t, while task offloading decisions $\delta(\tau)$, model selection $\gamma(\tau)$, and computing resource allocation decisions $f(\tau)$ are made at each time slot $\tau \in [t, t+T-1]$.

The upper bound of the drift-plus-penalty term can be calculated as

$$\begin{aligned}&\Delta_T(Q(t)) + V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \\ &\le \hat{B}T + \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} Q_{v,s}(\tau)\big(A_{v,s}(\tau) - W_{v,s}(\tau)\big)\right. \\ &\quad \left. + V\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \end{aligned} \tag{22}$$

where $\hat{B} = (1/2)\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}(A^2_{\max}(\tau) + W^2_{\max}(\tau))$. The detailed proof can be found in the Appendix.

The minimization of P_0 can be transformed into the minimization of the upper bound of the drift-plus-penalty in each time frame t. Since $\hat{B}$ is a constant at each time frame t, the short-term optimization problem P_1 is as follows:

$$\begin{aligned}\text{P1:} \min_{x(t),\delta(\tau),\gamma(\tau),f(\tau)} \quad & F1 = \sum_{\tau=t}^{t+T-1}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} Q_{v,s}(\tau)(A_{v,s}(\tau) \\ & \qquad -W_{v,s}(\tau)) + VE(\tau) \\ \text{s.t.} \quad & \text{C0-1, C0-2.}\end{aligned} \tag{23}$$

B. Optimization for Computing Resource Allocation

Analyzing the structure of P1, we observe that once $x(t)$, $\delta(\tau)$, and $\gamma(\tau)$ are fixed, the computing resource allocation problem in P1 can be simplified into one-shot optimization subproblems, denoted as P1-1. Furthermore, upon further simplification, the optimal solution for f can be derived analytically

$$\begin{aligned}\text{P1-1:} \min_{f(\tau)} \quad & F1(\tau) = Q_{v,s}(\tau)\big(A_{v,s}(\tau) - W_{v,s}(\tau)\big) \\ & \qquad +VE(\tau) \\ & = B(\tau) - \sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} G(\tau) f_{v,s}(\tau) \\ & \qquad + \sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} Z(\tau)\big[f_{v,s}(\tau)\big]^2 \\ \text{s.t.} \quad & (13)\text{–}(18) \\ & \begin{cases} B(\tau) = V\sum_{i\in\mathcal{I}}\sum_{s\in\mathcal{S}}\sum_{v\in\mathcal{V}}\Big[\delta_{i,s,v}(\tau)\frac{\varepsilon_i^d h_{src_i,v} d_{i,s}(\tau)}{R_{src_i,v}(\tau)} \\ \qquad +(1-\omega_s)\varepsilon_v^{\text{cold}} t_{s,v}^{\text{cold}}(\tau)\Big] \\ \qquad + \sum_{i\in\mathcal{I}}\sum_{s\in\mathcal{S}}\sum_{v\in\mathcal{V}}\sum_{b\in\mathcal{B}} Q_{v,s}(\tau)\delta_{i,s,v}(\tau)\gamma_{i,s,b}(\tau) \\ \qquad \big(d_{i,s}(\tau)/D_s\big)F\big(m_{s,b}\big), \\ G(\tau) = Q_{v,s}(\tau)x_{v,s}(t), \\ Z(\tau) = V\sum_{i\in\mathcal{I}}\sum_{b\in\mathcal{B}}\big[x_{v,s}(t)\delta_{i,s,v}(\tau)\gamma_{i,s,b}(\tau)\varepsilon_v^c \\ \qquad \big(d_{i,s}(\tau)/D_s\big)F\big(m_{s,b}\big)\big]. \end{cases}\end{aligned} \tag{24}$$

Equation (18) can be simplified into a constraint on the range of values for f, which can be expressed as follows:

$$\begin{aligned} & f_{\min} \le f_{v,s}(\tau) \le f_{\max} \quad \forall v \in \mathcal{V} \quad \forall s \in \mathcal{S} \\ & f_{\min} = \max_{i\in\{1,2,\ldots,|\mathcal{I}|\}} \left\{\frac{H(\tau)}{t_{i,s}^{\text{th}} - Y(\tau)}\right\} \\ & f_{\max} = f_v(\tau) \end{aligned}$$

$$\begin{cases} Y(\tau) = \sum_{v\in\mathcal{V}}\delta_{i,s,v}(\tau)\Big[\frac{h_{src_i,v} d_{i,s}(\tau)}{R_{src_i,v}(\tau)} + T_{v,s}^{qu.\max}\Big] \\ \qquad +(1-\omega_s)\cdot t_{s,v}^{\text{cold}}(\tau) \\ H(\tau) = \sum_{v\in\mathcal{V}}\sum_{b\in\mathcal{B}} x_{v,s}(t)\delta_{i,s,v}(\tau)\gamma_{i,s,b}(\tau)\big(d_{i,s}(\tau)/D_s\big)F\big(m_{s,b}\big). \end{cases} \tag{25}$$

Once the variables $x(t)$, $\delta(\tau)$, and $\gamma(\tau)$ are determined, $B(\tau)$, $G(\tau)$, $Z(\tau)$, $Y(\tau)$, and $H(\tau)$ become constants. It can be proven that both (13) and (18) are convex functions. Consequently, the transformed problem P1-1 is a convex problem, which can be solved using the Lagrangian dual decomposition method. The corresponding Lagrange function is presented in (26), as shown at the bottom of the next page.

Let $\lambda = [\lambda_1, \lambda_v, \ldots, \lambda_V]$, $\mu^{\min} = [\mu_{1,1}^{\min}, \mu_{v,s}^{\min}, \ldots, \mu_{|\mathcal{V}|,|\mathcal{S}|}^{\min}]$, and $\mu^{\max} = [\mu_{1,1}^{\max}, \mu_{v,s}^{\max}, \ldots, \mu_{|\mathcal{V}|,|\mathcal{S}|}^{\max}]$ represent the Lagrange multipliers. By applying KKT conditions and taking the derivative of $\mathcal{L}(f, \lambda, \mu)$ with respect to $f_{v,s}$, the optimal computing capacity allocation strategy is derived as follows:

$$f_{v,s}(\tau) = \frac{G(\tau) - \lambda_v + \mu_{v,s}^{\min} - \mu_{v,s}^{\max}}{2Z(\tau)}. \tag{27}$$

Utilizing the subgradient method, the solution gradually approaches one that satisfies the KKT conditions. The Lagrange multipliers in the ℓth iteration are updated as follows:

$$f_{v,s}^{(\ell+1)}(\tau) = \frac{G(\tau) - \lambda_v^{(\ell)} + \mu_{v,s}^{\min,(\ell)} - \mu_{v,s}^{\max,(\ell)}}{2Z(\tau)} \tag{28}$$

$$\lambda_v^{(\ell+1)} = \left[\lambda_v^{(\ell)} + \beta^{(\ell)}\left(\sum_{s\in\mathcal{S}} f_{v,s}^{(\ell+1)}(\tau) - f_v(\tau)\right)\right]^+ \tag{29}$$

$$\mu_{v,s}^{\min,(\ell+1)} = \left[\mu_{v,s}^{\min,(\ell)} + \beta^{(\ell)}\left(f_{\min} - f_{v,s}^{(\ell+1)}(\tau)\right)\right]^+ \tag{30}$$

$$\mu_{v,s}^{\max,(\ell+1)} = \left[\mu_{v,s}^{\max,(\ell)} + \beta^{(\ell)}\left(f_{v,s}^{(\ell+1)}(\tau) - f_{\max}\right)\right]^+ \tag{31}$$

where β denotes the step sizes, and $[x]^+$ represents the projection of x onto the nonnegative quadrant, which is defined as $\max\{x, 0\}$.

Leveraging the KKT conditions, the optimal computing resource allocation $f(\tau)$ can be directly obtained, reducing the action space of the MARL model. When $x(t)$, $\delta(\tau)$, and $\gamma(\tau)$ are solved using MARL algorithm, the KKT conditions can serve as an optimization subroutine to efficiently determine the optimal computing resource allocation $f(\tau)$.

C. Hierarchical Markov Decision Process Framework for Two Time-Scale Optimization

Considering the two time-scale nature of the P1 problem, hierarchical reinforcement learning (HRL) with an options framework has been proven to be particularly effective [30], [31]. The options framework extends traditional RL by introducing temporally extended actions, called options, which consist of a policy, a termination condition, and an initiation set [32]. This allows agents to operate at different time scales, making it well-suited for multiscale decision-making problems. Therefore, an HMDP framework is designed to address this problem, where the upper layer focuses on long-term service placement decisions $x(t)$, and the lower layer handles real-time decisions for task node selection $\delta(\tau)$

and model selection $\gamma(\tau)$. In a cooperative setting, agents collaboratively optimize their policies to maximize long-term rewards, leveraging shared rewards r_{upper} and r_{lower} across both the upper and lower layers.

The upper layer is modeled as a multiagent semi-Markov decision process (MAS-SMDP)

$$\Gamma_{\text{upper}} = \left(\mathcal{S}_{\text{upper}}, \{\mathcal{O}_v\}_{v=1}^{|\mathcal{V}|}, r_{\text{upper}}, p_{\text{upper}}, \mathcal{T}_{\text{upper}}, \gamma_{\text{upper}}\right) \quad (32)$$

where $\mathcal{S}_{\text{upper}}$ is the global state space of the upper layer. $\mathcal{O}_v$ is the option space of computing node v, representing placement strategies. $|\mathcal{V}|$ denotes the total number of computing nodes. r_{upper} is the long-term reward function. p_{upper} is the state transition probability. $\mathcal{T}_{\text{upper}}$ is the option duration, and γ_{upper} is the discount factor.

The lower layer is modeled as a multiagent Markov decision process (MAS-MDP)

$$\Gamma_{\text{lower}} = \left(\mathcal{S}_{\text{lower}}, \{\mathcal{A}_i\}_{i=1}^{|\mathcal{I}|}, r_{\text{lower}}, p_{\text{lower}}, \gamma_{\text{lower}}\right) \quad (33)$$

where $\mathcal{S}_{\text{lower}}$ is the global state space of the lower layer. $\mathcal{A}_i$ is the action space of terminal i, including task offloading node and model selection. $|\mathcal{I}|$ denotes the total number of terminals. r_{lower} is the immediate reward. p_{lower} is the state transition probability, and γ_{lower} is the discount factor.

In the hierarchical framework, the upper and lower layers collaborate through state sharing, action guidance, and reward feedback. The upper layer optimizes the service placement strategy, which guides the task node and model selection decisions of the lower layer. The lower layer's state space includes information, such as queue length and computing capacity, reflecting the impact of the upper-layer strategy. Meanwhile, the lower layer executes real-time tasks and provides feedback on energy consumption and Lyapunov drift, which is aggregated and sent to the upper layer to optimize long-term service placement. The upper layer focuses on long-term goals, such as energy consumption and Lyapunov drift over a time frame, while the lower layer addresses short-term goals, including energy consumption, Lyapunov drift, and QoS satisfaction within a time slot. This feedback allows the system to dynamically adapt to IoT environment changes and achieve global optimization.

V. HMFD3QN Algorithm Design

As the number of computing nodes and terminals increases, MARL faces scalability challenges, such as poor convergence and high communication overhead. Mean field theory [33] approximates multiagent problems into a two-agent setting, where one agent represents the collective effect of the population (i.e., the mean field distribution), thereby significantly reducing computational complexity. Previous works [22], [24], [34], and [35] have predominantly relied on centralized learning methods, where all agents learn and update a shared centralized policy. However, in the scenario discussed in this article, the upper-layer computing nodes exhibit heterogeneous computing and storage capacities, while the lower-layer terminals have diverse task requests and local observations, which violate the assumptions of homogeneity and indistinguishability in mean field games.

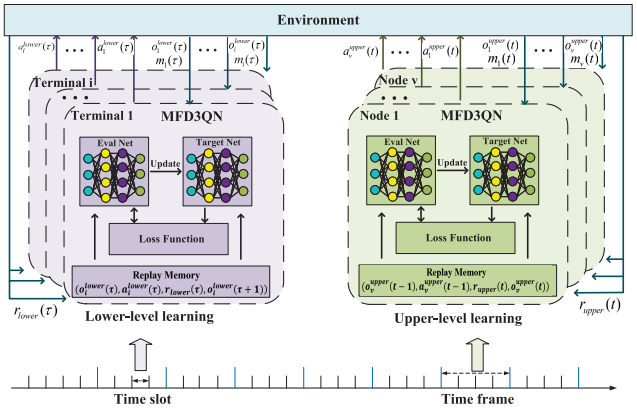


Fig. 3. HMFD3QN algorithm framework.

To address this issue, this article adopts the decentralized mean field game (DMFG) approach [36], relaxing the aforementioned assumptions. This allows agents to possess independent characteristics and develop their own distinct policies. In DMFG, the availability of immediate global mean field is no longer assumed. Instead, agents access only local information and employ neural networks to effectively model the mean field during the training process. Meanwhile, the assumption that each agent's influence on the environment is infinitesimal is retained, allowing agents to focus solely on computing their best response to the mean field.

Based on the hierarchical framework and DMFG, the HMFD3QN algorithm is designed to address two time-scale complex optimization problems, as illustrated in Fig. 3. At the upper layer, each computing node acts as an agent and deploys an MFD3QN algorithm to optimize AI service placement decisions. At the lower layer, each terminal serves as an agent and deploys its own MFD3QN algorithm to optimize node and model selection decisions.

A. Upper-Level Learning for Long-Term Optimization

Local Observation: At time frame t, the global state space $\mathcal{S}_{\text{upper}}$ is composed of the local observations of each computing node, which can be expressed as

$$\mathcal{L}(f,\lambda,\mu) = B(\tau) - \sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} G(\tau) f_{v,s}(\tau) + \sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} Z(\tau)\left[f_{v,s}(\tau)\right]^2 + \sum_{v\in\mathcal{V}}\lambda_v\left(\sum_{s\in\mathcal{S}} f_{v,s}(\tau) - f_v(\tau)\right) + \sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\mu_{v,s}^{\min}\left(f_{\min} - f_{v,s}(\tau)\right) + \sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\mu_{v,s}^{\max}\left(f_{v,s}(\tau) - f_{\max}\right) \quad (26)$$

$$\mathcal{S}_{\text{upper}} = \left\{o_1^{\text{upper}}(t), o_2^{\text{upper}}(t), \ldots, o_{|\mathcal{V}|}^{\text{upper}}(t)\right\} \tag{34}$$

$$o_v^{\text{upper}}(t) = \left\{\mathbf{x}_v(t-1), \boldsymbol{\phi}_v(t-1)\right\} \tag{35}$$

$$\mathbf{x}_v(t-1) = \left\{x_{v,1}(t-1), x_{v,2}(t-1), \ldots, x_{v,|\mathcal{S}|}(t-1)\right\} \tag{36}$$

$$\boldsymbol{\phi}_v(t-1) = \left\{\phi_{v,1}(t-1), \phi_{v,2}(t-1), \ldots, \phi_{v,|\mathcal{S}|}(t-1)\right\} \tag{37}$$

where $o_v^{\text{upper}}(t)$ represents the local observation of computing node v at time frame t. The term $\mathbf{x}_v(t-1)$ encapsulates the service placement strategy of node v at the previous time frame $t-1$. Meanwhile, $\boldsymbol{\phi}_v(t-1)$ is a vector representing the popularity of service requests received by node v over the preceding T time slots. The popularity of service s $\phi_{v,s}(t-1)$ is defined as the ratio of the total number of requests for service s to the total number of requests received by node v at the previous time frame $t-1$. It is expressed as

$$\phi_{v,s}(t-1) = \frac{\sum_{\tau=t-T+1}^{t}\sum_{i=0}^{|\mathcal{I}|}\delta_{i,s,v}(\tau)\left(d_{i,s}(\tau)/D_s\right)}{\sum_{\tau=t-T+1}^{t}\sum_{s=0}^{|\mathcal{S}|}\sum_{i=0}^{|\mathcal{I}|}\delta_{i,s,v}(\tau)\left(d_{i,s}(\tau)/D_s\right)}. \tag{38}$$

Option Action: The action taken by the upper-level agent can be expressed as

$$a_v^{\text{upper}}(t) = \{\mathbf{x}_v(t)\} \tag{39}$$

where $\mathbf{x}_v(t) = \{x_{v,1}(t), x_{v,s}(t), \ldots, x_{v,|\mathcal{S}|}(t)\}$.

Reward: At the start of each time frame, all lower-level agents take actions based on the observed service placement strategy. Once the time frame concludes, the energy consumption and the Lyapunov drift corresponding to the T time slots within that frame are reported back. These feedback values are then used to compute the reward for the upper-level agent, specifically

$$r_{\text{upper}}(t) = -F1(t) \tag{40}$$

where $F1(t)$ serves as the optimization objective of (23) at time frame t.

In the upper-level learning framework, the Q-function for each computing node v in MARL is approximated by decomposing global interactions into pairwise local interactions, as follows:

$$Q_v\left(o_v^{\text{upper}}(t), a(t)\right) = \frac{1}{|\mathcal{N}_v|}\sum_{v'\in\mathcal{N}_v} Q_v\left(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), a_{v'}^{\text{upper}}(t)\right) \tag{41}$$

where $a(t)$ represents the joint influence of all agents' actions, and $\mathcal{N}_v$ denotes the set of neighboring nodes for node v. Additionally, $a_{v'}^{\text{upper}}(t)$ refers to the service placement strategy of a neighboring node v' at time frame t. Using Taylor's theorem, the Q-function can be expressed as [22]

$$\begin{aligned} Q_v\left(o_v^{\text{upper}}(t), a(t)\right) &= Q_v\left(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), m_v(t)\right) \\ &+ \frac{1}{2|\mathcal{N}_v|}\sum_{v'\in\mathcal{N}_v} R_v\left(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), a_{v'}^{\text{upper}}(t)\right) \\ &\approx Q_v\left(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), m_v(t)\right) \end{aligned} \tag{42}$$

where $m_v(t)$ is the mean field of neighboring nodes' actions, and $R_v(\cdot)$ represents the Taylor remainder. By retaining only the first-order term, the Q-function simplifies to: $Q_v(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), m_v(t))$. This formulation transforms complex interactions between node v and its neighbors into a simplified interaction with a virtual mean-field agent, significantly reducing computational complexity while preserving key local interactions.

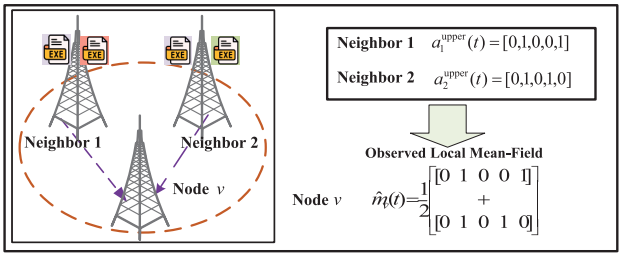


Fig. 4. Observed local mean-field example with five types of AI services and two neighbors, where each neighbor can choose which services to place, represented using one-hot encoding (1 for placed, 0 for not placed).

The mean-field approximation is modeled by a neural network NN_v, taking the current state $o_v^{\text{upper}}(t)$ and the previous observed local mean-field action $\hat{m}_v(t-1)$ as inputs [36]

$$m_v(t) = \text{NN}_v\left(o_v^{\text{upper}}(t), \hat{m}_v(t-1)\right). \tag{43}$$

The network NN_v is implemented as a fully connected architecture with two hidden layers (50 nodes each, ReLU activation) and a softmax output layer. It is trained using the mean-square error (MSE) between the predicted mean-field action $m_v(t)$ and the observed local mean field $\hat{m}_v(t)$. The observed local mean field $\hat{m}_v(t)$ is defined as the expectation of the service placement strategy of the neighboring nodes of node v at time frame t

$$\hat{m}_v(t) = \mathbb{E}_{v'\in\mathcal{N}_v}\left[a_{v'}^{\text{upper}}(t)\right]. \tag{44}$$

Fig. 4 illustrates an example of the observed local mean-field for AI service placement. The observed local mean field of node v is defined as the average of the placement strategies of its two neighboring nodes.

Each node selects its action based on a Boltzmann policy π_v^{upper}, as shown as follows:

$$\begin{aligned} &\pi_v^{\text{upper}}\left(a_v^{\text{upper}}(t)|o_v^{\text{upper}}(t), m_v(t)\right) \\ &= \frac{\exp\left(\zeta Q_v\left(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), m_v(t)\right)\right)}{\sum_{a_v^{\text{upper}'}\in\mathcal{O}_v}\exp\left(\zeta Q_v\left(o_v^{\text{upper}}(t), a_v^{\text{upper}'}, m_v(t)\right)\right)} \end{aligned} \tag{45}$$

where $a_v^{\text{upper}'} \in \mathcal{O}_v$ represents a possible action from the option action space $\mathcal{O}_v$ of node v. ζ is a temperature parameter that controls the tradeoff between exploration (selecting less certain actions) and exploitation (choosing actions with higher Q-values).

To approximate the Q-function for each agent, the MFD3QN framework is employed. This method integrates the strengths of mean-field approximation, double Q-learning, and the dueling network architecture, making it particularly effective for high-dimensional environments with complex agent interactions. To enhance learning stability and efficiency, the Q-function is decomposed into two components:

1) the state-value function $V(o_v^{\text{upper}}(t), m_v(t))$, which represents the overall value of a state and 2) the advantage function $A(a_v^{\text{upper}}(t)|o_v^{\text{upper}}(t), m_v(t))$, which captures the relative importance of actions in the given state. This decomposition is expressed as

$$Q_v\big(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), m_v(t)\big) = V\big(o_v^{\text{upper}}(t), m_v(t)\big) + A\big(a_v^{\text{upper}}(t)|o_v^{\text{upper}}(t), m_v(t)\big). \tag{46}$$

To address the overestimation bias commonly encountered in Q-learning, MFD3QN employs double Q-learning, which maintains two separate networks: 1) the evaluation network (Q_{eval}) for selecting actions and 2) the target network (Q_{target}) for evaluating the selected actions. The target value is defined as

$$y = r_{\text{upper}}(t) + \gamma_{\text{upper}} Q_{\text{target}}\big(o_v^{\text{upper}}(t+1) \arg\max_{a_v^{\text{upper}}(t+1)} Q_{\text{eval}}\big(o_v^{\text{upper}}(t+1), a_v^{\text{upper}}(t+1), m_v(t+1)\big) m_v(t+1)\big). \tag{47}$$

The training process minimizes the following loss function:

$$\mathcal{L} = \mathbb{E}\Big[\big(Q_{\text{eval}}\big(o_v^{\text{upper}}(t), a_v^{\text{upper}}(t), m_v(t)\big) - y\big)^2\Big] \tag{48}$$

where the expectation is over transitions sampled from the replay buffer. The replay buffer helps to break the temporal correlation in the data, stabilizing the learning process.

To further improve training stability, the target network Q_{target} is periodically updated using the weights of the evaluation network Q_{eval}. The update rule is defined as

$$\theta_{\text{target}} \leftarrow \alpha\theta_{\text{eval}} + (1-\alpha)\theta_{\text{target}} \tag{49}$$

where θ_{eval} and θ_{target} are the parameters of the evaluation and target networks, respectively, and $\alpha \in [0, 1]$ is the soft update coefficient. This gradual update ensures that the target network evolves smoothly, reducing training instability caused by sudden parameter changes.

B. Lower-Level Learning for Short-Term Optimization

Local Observation: At time slot τ, the global state space $\mathcal{S}_{\text{lower}}$ is composed of the local observations from each terminal, which can be expressed as

$$\mathcal{S}_{\text{lower}} = \Big\{o_1^{\text{lower}}(\tau), o_2^{\text{lower}}(\tau), \ldots, o_{|\mathcal{I}|}^{\text{lower}}(\tau)\Big\} \tag{50}$$

$$o_i^{\text{lower}}(\tau) = \Big\{P_{i,s}(\tau), \big\{|Q_{v,s}(\tau-1)|\big\}_{v=1}^{|\mathcal{V}|}, \big\{f_{v,s}(\tau-1)\big\}_{v=1}^{|\mathcal{V}|}\Big\} \tag{51}$$

where $o_i^{\text{lower}}(\tau)$ represents the local observation of terminal i at time slot τ. At time slot τ, the service type requested by each terminal is known. To eliminate redundancy, $o_i^{\text{lower}}(\tau)$ includes only the information relevant to service s. $P_{i,s}(\tau)$ denotes the set of task requests for AI service s from terminal i at time slot τ, as defined in (2). $|Q_{v,s}(\tau-1)|$ represents the queue length for service s at computing node v in the previous time slot $\tau-1$. $f_{v,s}(\tau-1)$ denotes the computing capability allocated by computing node v to service s.

Action: The action of the lower-level agent can be expressed as

$$a_i^{\text{lower}}(\tau) = \{\boldsymbol{\delta}_{i,s}(\tau), \boldsymbol{\gamma}_{i,s}(\tau)\} \tag{52}$$

where $\boldsymbol{\delta}_{i,s}(\tau) = \{\delta_{i,s,1}(\tau), \delta_{i,s,2}(\tau), \ldots, \delta_{i,s,|\mathcal{V}|}(\tau)\}$. $\boldsymbol{\gamma}_{i,s}(\tau) = \{\gamma_{i,s,1}(\tau), \gamma_{i,s,2}(\tau), \ldots, \gamma_{i,s,|\mathcal{B}|}(\tau)\}$.

Reward: The reward for the lower-level agent is defined by the optimization objective $F1(\tau)$ of (23) at time slot τ, along with a soft penalty for tasks that fail to meet the QoS requirements. It is formulated as

$$r_{\text{lower}}(\tau) = -\Big[F1(\tau) + \omega_{q1} e^{\omega_{q2} N_{\text{QoS}}(\tau)}\Big] \tag{53}$$

where ω_{q1} and ω_{q2} are weight factors associated with the QoS penalty. $N_{\text{QoS}}(\tau)$ denotes the number of tasks whose QoS requirements are not satisfied at time slot τ. The exponential function $e^{N_{\text{QoS}}(\tau)}$ introduces a rapidly increasing penalties as $N_{\text{QoS}}(\tau)$ grows, ensuring that the system prioritizes satisfying QoS requirements for tasks.

To enhance scalability, the lower-level learning employs MFD3QN, which approximates the Q-function $Q_i(o_i^{\text{lower}}(\tau), a_i^{\text{lower}}(\tau), m_i(\tau))$ through a mean-field approach, capturing the collective interaction behavior among neighboring terminals.

C. Algorithm Procedure

Algorithm 1 begins by initializing the environment, including computing nodes, terminals, communication links, and related parameters. It also initializes the Q networks and replay buffers for both the upper-level (computing nodes) and lower-level (terminals) agents.

During each time frame t, the upper level (computing nodes) observes the system state, estimates the mean field of deployment actions from neighboring nodes, and selects AI service placement strategies (lines 9–13). Since the upper level operates at the scale of a time frame, it only receives the reward from the previous time frame $t-1$ after completing T time slots in the current time frame t. The selected placement strategies define the service environment for the lower level.

Within each time frame, at every time slot τ (a single time slot), terminals at the lower level observe their local states, estimate the mean field actions, and select the optimal node and model based on the observed state and the estimated mean field actions (lines 19–24). The optimal solution for computing resource allocation is derived based on the KKT conditions. Terminals then execute their selected actions, receive rewards based on their performance, and store their experiences in the replay buffer.

Both the upper and lower levels periodically sample mini-batches from their respective replay buffers to update the parameters of their MFD3QN models (lines 15, 26). This hierarchical structure efficiently decouples the optimization tasks of computing nodes and terminals while capturing their interactions through mean-field approximations.

D. Complexity Analysis

The complexity of computing resource allocation strategy based on the KKT conditions is primarily determined by the

Algorithm 1: HMFD3QN Algorithm

Initialize the environment, Q networks $Q_\theta^{\text{upper},v}$, $Q_\theta^{\text{lower},i}$, target networks, and replay buffers $\mathcal{D}_{\text{upper}}$, $\mathcal{D}_{\text{lower}}$;
for *episode* $e = 1, 2, \ldots, E$ **do**
  Reset the environment and compute neighbors for nodes and terminals;
  Update exploration rate ϵ;
  for *time slot* $\tau = 1, 2, \ldots, \mathcal{T}$ **do**
    if $\tau \mod T = 0$ *(time frame t)* **then**
      // Upper-level decision at time frame t;
      for *each computing node* $v = 1, \ldots, |\mathcal{V}|$ **do**
        Observe $o_v^{\text{upper}}(t)$, estimated mean field action $m_v(t-1)$;
        Select action $\mathbf{x}_v(t)$ using $Q_\theta^{\text{upper},v}$, defined in (45) or random exploration with probability ϵ;
        Execute action, receive reward $r_{\text{upper}}(t-1)$, and store transition in $\mathcal{D}_{\text{upper}}$;
        Obtain the current observed local mean action $\hat{m}_v(t)$;
        Update the mean field network using a MSE$(\hat{m}_v(t), m_v(t))$;
      end
      Update Q_θ^{upper} for all nodes using mini-batches from $\mathcal{D}_{\text{upper}}$, following (48);
    end
    // Lower-level decision at time slot τ;
    for *each terminal* $i = 1, \ldots, |\mathcal{I}|$ **do**
      Observe $o_i^{\text{lower}}(\tau)$, estimated mean field action $m_i(\tau-1)$;
      Select action $a_i^{\text{lower}}(\tau)$ using $Q_\theta^{\text{lower},i}$, defined in (45) or random exploration with probability ϵ;
      Obtain $f(\tau)$ by KKT-conditions ;
      Execute action, receive reward $r_{\text{lower}}(\tau)$, and store transition in $\mathcal{D}_{\text{lower}}$;
      Obtain the current observed local mean action $\hat{m}_i(\tau)$;
      Update the mean field network using a MSE$(\hat{m}_i(\tau), m_i(\tau))$;
    end
    Update Q_θ^{lower} for all terminals using mini-batches from $\mathcal{D}_{\text{lower}}$, following (48);
  end
end
Deploy trained Q_θ *models for decentralized decision-making*;

computation of resource allocation $f_{v,s}(\tau)$ and the updates of the Lagrange multipliers λ_v, $\mu_{v,s}^{\min}$, and $\mu_{v,s}^{\max}$. The complexity per iteration is $O(|\mathcal{V}| \cdot |\mathcal{S}|)$, where $|\mathcal{V}|$ is the number of computing nodes and $|\mathcal{S}|$ is the number of service type. Assuming a maximum of L iterations, the total complexity is $O(L \cdot |\mathcal{V}| \cdot |\mathcal{S}|)$.

The HMFD3QN algorithm significantly reduces computational complexity by employing mean-field theory. In traditional MARL, modeling the joint actions of all agents results in exponential complexity, expressed as $O(n_{a^{\text{upper}}}^{|\mathcal{V}|} + n_{a^{\text{lower}}}^{|\mathcal{I}|})$, where $n_{a^{\text{upper}}}$ and $n_{a^{\text{lower}}}$ are the action space sizes for upper- and lower-level agents, and $|\mathcal{V}|$ and $|\mathcal{I}|$ represent the numbers of upper- and lower-level agents, respectively. By approximating the collective actions of other agents as a mean-field action, HMFD3QN simplifies decision-making. The complexity is reduced to $O(|\mathcal{V}|^2 n_{a^{\text{upper}}})$ at the upper level and $O(|\mathcal{I}|^2 n_{a^{\text{lower}}})$ at the lower level, giving an overall complexity of $O(|\mathcal{V}|^2 n_{a^{\text{upper}}} + |\mathcal{I}|^2 n_{a^{\text{lower}}})$. In sparse scenarios, where each agent interacts with only a few neighbors, the complexity further decreases to $O(|\mathcal{V}| n_{a^{\text{upper}}} + |\mathcal{I}| n_{a^{\text{lower}}})$, making it scalable for large systems with localized interactions.

The HMFD3QN algorithm computes the decision variables $x(t)$, $\delta(\tau)$, and $\gamma(\tau)$, while the KKT conditions are employed to optimize the resource allocation $f(\tau)$ based on these outputs. Since the two steps are executed sequentially, the overall computational complexity is determined by the more computationally intensive of the two components. In sparse scenarios, this complexity is expressed as $O(\max(L \cdot |\mathcal{V}| \cdot |\mathcal{S}|, |\mathcal{V}| n_{a^{\text{upper}}} + |\mathcal{I}| n_{a^{\text{lower}}}))$.

In DMFG, each agent i requires, in addition to the Q-network Q_i used for outputting actions, an additional neural network m_i to approximate the mean-field actions, which indeed introduces some computational complexity. This article only employs a fully connected neural network to approximate the mean field action, thereby effectively controlling the computational cost. As shown in Fig. 5, we computed the number of trainable parameters in the mean-field approximation networks m_i for all agents and their proportion r relative to the total number of trainable parameters in all networks (including Q_i and m_i for all agents) under scenarios with varying numbers of agents. The figure shows that the parameter proportion r of the mean-field approximation networks consistently remains below 33%, and the number of network parameters exhibits a linear growth trend. Thus, the added computational cost remains acceptable.

To enhance the decision-making and deployment efficiency of the algorithm, a network digital twin (NDT) layer can be integrated into the E-N-C system [37], [38]. By leveraging historical data to construct virtual replicas of physical systems, the NDT spans the entire network lifecycle, including planning, deployment, operation, and optimization. Furthermore, NDT modeling allows the feasibility of network designs and configurations to be validated in advance, ensuring optimal performance.

VI. Simulation Results and Analysis

This section presents simulation experiments conducted to evaluate the performance of the proposed algorithm. The simulation setup and corresponding numerical results are detailed as follows.

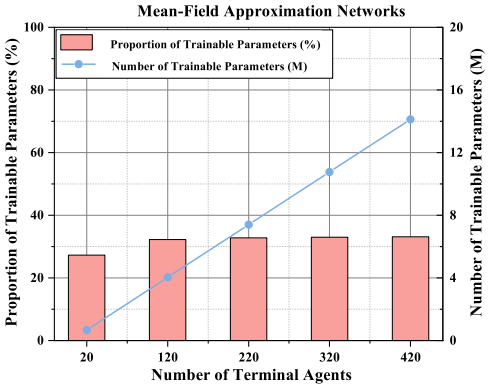
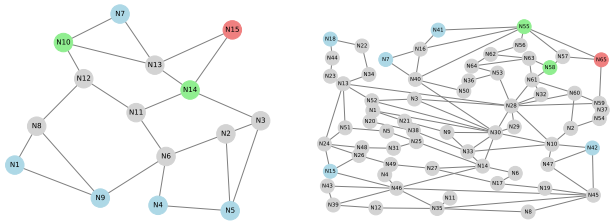


Fig. 5. Computational Cost Analysis of Mean-Field Approximation Networks. The bar chart illustrates the number of trainable parameters in the mean-field approximation networks for all agents as a proportion r of the total number of trainable parameters across all networks (including Q_i and m_i for all agents) under scenarios with varying numbers of agents. The line chart, on the other hand, depicts the trend in the total number of trainable parameters in the mean-field approximation networks for all agents.



(a) (b)

Fig. 6. Network topology for simulations. (a) Altanta. (b) Ta2.

TABLE II
NETWORK PARAMETERS

Parameter	Value
Transmission rate $R_{i,j}(\tau)$	$[50, 100]$ MB/s
Computing capacity (E-N-C)	$[3000, 4000]$, $[800, 1200]$, 5600 GFLOPS
RAM Storage capacity (E-N-C)	$20, 10, 80$ GB
HDD Storage capacity (E-N-C)	$10, 10, 80$ GB
Service requests per terminal	20 batchs/τ
Transmission energy coefficient ε_i^d	0.2 W
Computation energy coefficient ε_v^c	5×10^{-10} W
Cold-start energy coefficient ε_v^{cold}	0.2 W
Cold-start time $t_{s,v}^{cold}$	$[0.15, 0.85]$ s
Lyapunov-energy balancing coefficient V	1×10^{-7}

TABLE III
LEARNING PARAMETERS

Parameter	Value
Neural network architecture	D3QN [40]
Upper-level Agent number[a]	7
Lower-level Agent number	60
Number of training episode	3000
Number of testing episode	10
Time step of each episode	100
AI service placement interval	10
QoS weight factor ω_{q1}, ω_{q2}	$1000, 0.12$
Upper-level Adam learning rate	0.0001
Lower-level Adam learning rate	0.0001
Discount factor	0.99
ϵ annealing length	1500
Hidden size	$128, 64$
Replay buffer size	3×10^5
Activation function	ReLU
Batch size	64

[a] Since the cloud server hosts all AI service types, placement strategies are unnecessary, resulting in 7 agents.

A. Simulation Settings

Experiments are conducted on the Atlanta network, which consists of 15 nodes and 22 links, and the Ta2 network, which consists of 65 nodes and 108 links, within the SNDlib framework [40]. The network structures are illustrated in Fig. 6. Each network contains eight computing nodes: five edge servers (blue nodes), one central cloud server (red nodes), and two network nodes (green nodes). These nodes have specific storage and computing capacities. Link transmission rates are randomly generated, and data is transmitted using the shortest-path mechanism. Nodes without computing capabilities (gray nodes) act as relay nodes to forward tasks. Terminals are randomly associated to edge servers, and their service preferences in each time slot follow a Zipf distribution with a shape parameter of 0.8 [19]. In RL, one step corresponds to one time slot, and ten steps correspond to one time frame. The network environment parameters are presented in Table II, while the learning parameters are detailed in Table III.

Five image-based AI service types commonly utilized in IoT applications are considered: image classification, semantic segmentation, object detection, instance segmentation, and video classification. Among these, object detection and instance segmentation are designated as occasional services. Each service incorporates 2 to 4 AI models, each characterized by its accuracy, storage requirements, computational demand (measured in GFLOPs), and input data size. Further details are available online.[1]

B. Baseline

We evaluate the performance of the proposed algorithm by comparing it against the following baselines.

1) *HIQL:* This algorithm employs an HRL structure, where each agent independently learns and makes decisions using the D3QN algorithm [39], without any collaboration or interaction among agents.
2) *HMADDPG:* This approach utilizes an HRL framework, with each layer implementing the MADDPG algorithm [41]. It features independent actor networks combined with a shared critic network, leveraging centralized training and decentralized execution to model agent collaboration.
3) *MFD3QN-GA:* This algorithm integrates MFD3QN in the upper layer for large time-scale optimization with a genetic algorithm in the lower layer for fine-grained resource allocation and optimization.
4) *POP-MFD3QN:* This method adopts a Popular caching scheme at the upper layer, where services are placed based on the popularity observed in the previous time

[1] https://github.com/zhangzhenBrave/HMFD3QN

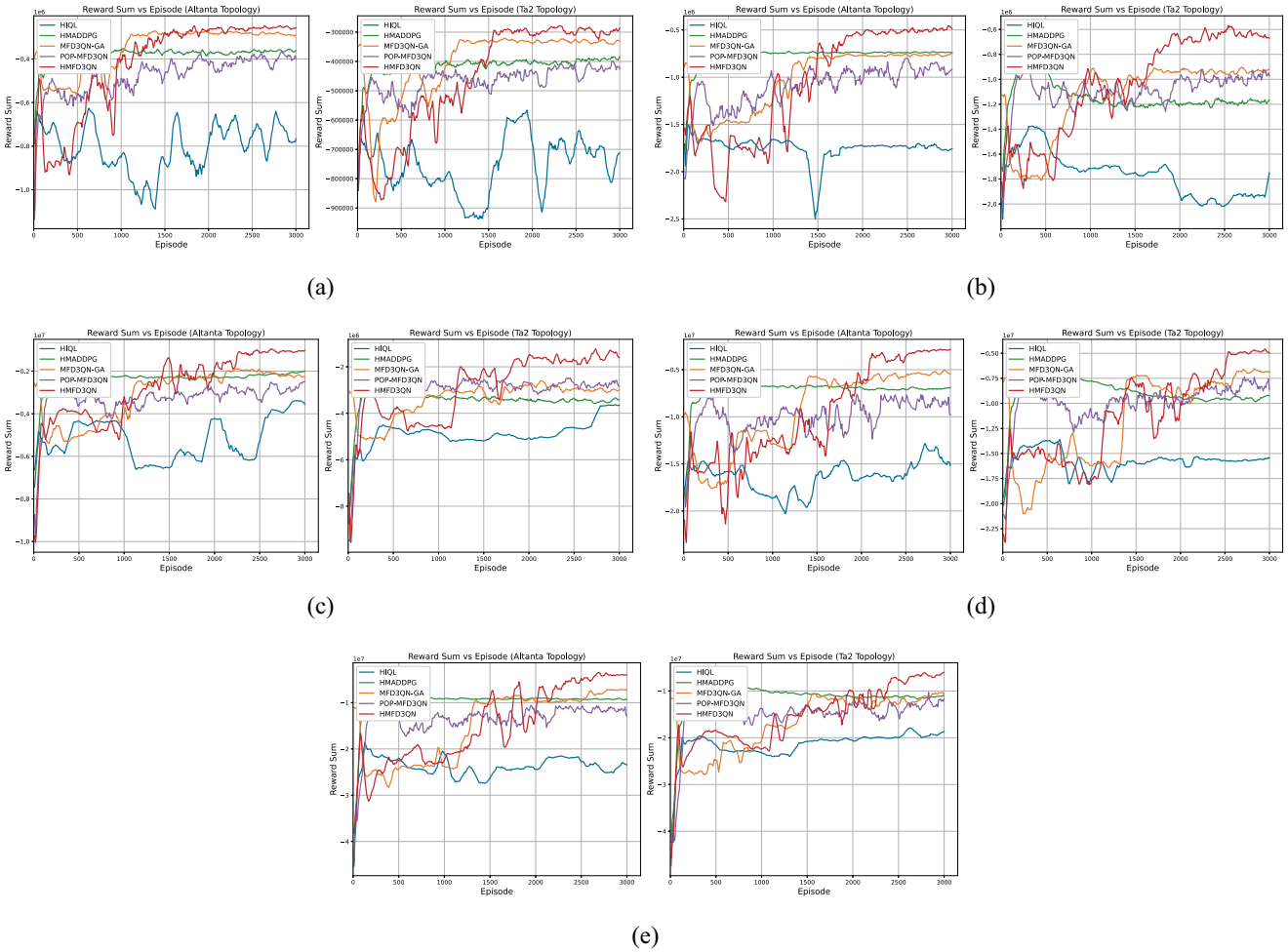


Fig. 7. Training reward performance in different terminal-scale scenarios. (a) 20 Terminals. (b) 60 Terminals. (c) 100 Terminals. (d) 300 Terminals, (e) 500 Terminals.

frame. The lower layer then utilizes the MFD3QN approach to handle task scheduling.

C. Training Results

Fig. 7 shows the training performance of five algorithms in scenarios with different numbers of terminals. On average across the two topologies, the reward improvement of HMFD3QN compared to the second-best algorithm is approximately 13% for 20 agents, 33% for 60 agents, and around 40% for 100, 300, and 500 agents. These results demonstrate that as the number of agents increases, HMFD3QN consistently achieves a significant advantage, particularly in large-scale scenarios with 100 or more agents. This is attributed to the HRL design, which enables the model to make decisions regarding AI service placement, task node and model selection across different time scales. Furthermore, the use of mean-field approximation reduces communication interactions between terminals, allowing the algorithm to converge to a better solution even in large-scale scenarios.

The HIQL algorithm, due to its lack of collaboration, fails to converge in cooperative scenarios where interagent coordination is essential. HMADDPG, relying solely on the shared Critic network for interaction, suffers from scalability issues in large-scale multiagent environments, leading to suboptimal performance. In contrast, the MFD3QN-GA algorithm benefits from MFD3QN for large time-scale optimization in the upper layer. However, as the number of terminals increases, the solution space grows rapidly, and the genetic algorithm's iterative search becomes slower, significantly reducing efficiency. The POP-MFD3QN algorithm adopts a static placement strategy in the upper layer, selecting the service placement for the current time frame based on the task offloading preferences from the previous time frame. However, this approach fails to capture the dynamic task offloading behavior of terminals in real-time.

D. Mean Field Approximation Error Analysis

In this section, we plot the boxplot of the per-agent mean field approximation error (measured using MSE) between the estimated mean field actions $m_i(\tau)$ (from the neural network NN_i representation) and the observed local mean field actions $\hat{m}_i(\tau)$ for varying numbers of terminal agents, as shown in Fig. 8. The blue dots represent the mean field approximation error for each agent at different agent numbers, collected after the neural network has converged. From Fig. 8, it is evident that the average approximation error decreases significantly as the number of agents increases. This supports

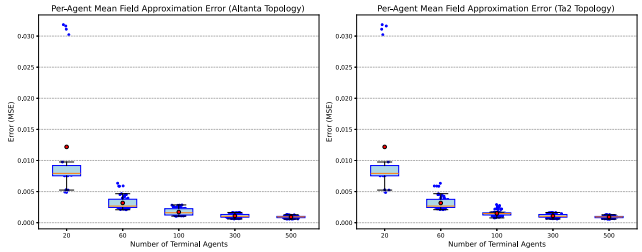


Fig. 8. Per-Agent Mean Field Approximation Error Across Different Numbers of Terminal Agents. (Red dots: mean, red line: median, blue dots: per-agent errors).

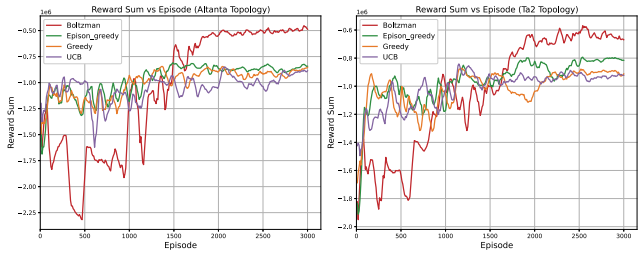


Fig. 9. Training reward performance in different policies (60 Terminals, 8 Computing Nodes).

the core assumption of mean field theory: when the number of agents is sufficiently large, the error between the local mean field and the true mean field becomes negligible. Notably, when the number of agents reaches 100 or more, the error stabilizes and approaches zero, demonstrating the system's convergence. Additionally, as the number of agents grows, the error distribution becomes increasingly concentrated, as reflected by the narrowing range of individual errors (blue dots) and the reduced height of the box plots. This indicates that the errors not only decrease but also become more stable and consistent across agents.

Moreover, the neural network effectively models the mean field across all tested agent numbers. Even in small-scale systems (e.g., 20 agents), the MSE remains relatively low (˜0.03). As the number of agents increases (e.g., 60, 100, 300, and 500), the MSE further decreases, reaching a near-negligible level. This highlights the robustness of the neural network in capturing mean field dynamics across different agent scales.

E. Performance Under Different Policies

This experiment evaluates the adaptability of different policies by comparing their performance under varying network topologies. As shown in Fig. 9, the results demonstrate that the Boltzman policy consistently achieves better performance compared to the other policies across both network topologies. Its probabilistic action selection mechanism enables a more adaptive balance between exploration and exploitation, which is crucial in dynamic and complex network environments. In contrast, the greedy policy, which focuses exclusively on exploitation, struggles in the early episodes due to insufficient exploration. Similarly, the ϵ-greedy policy shows improved exploration compared to the greedy policy but lacks the adaptability of the Boltzman policy due to its fixed exploration strategy. The upper confidence bound (UCB) policy [42] initially performs competitively but is less effective in long-term optimization, particularly in nonstationary scenarios.

F. Performance Under Different Load Conditions Scenarios

This experiment aims to assess how different algorithms perform under varying load conditions. As illustrated in Fig. 10, increasing the number of service requests per terminal intensifies the computational load on nodes, causing noticeable performance degradation in most algorithms. HIQL, HMADDPG, MFD3QN-GA, and POP-MFD3QN exhibit significant increases in energy consumption, task delays, and Lyapunov drift, along with a sharp decline in QoS satisfaction under high-load scenarios.

When service requests increased from 20 to 30, the Total Lyapunov Drift of HMADDPG decreased, primarily due to its reliance on a shared Critic Network. In HMADDPG, all agents share a single Critic Network to evaluate the value of joint states and actions. However, this design faces significant scalability and stability challenges in large-scale multiagent environments. As the number of agents and task complexity grow, the Critic Network encounters high-dimensional inputs and greater nonstationarity caused by changing agent policies, making adaptation and accurate evaluation more difficult. This instability can lead HMADDPG to fall into suboptimal solutions when service requests are at 20, resulting in performance degradation.

By contrast, HMFD3QN demonstrates remarkable robustness even as the load escalates. Under the heaviest network load, HMFD3QN surpasses the second-best algorithm by reducing energy consumption by 34%, decreasing Lyapunov drift by 12%, improving QoS task satisfaction by 17%, and cutting average task delay by 12%. The HRL mechanism employed by HMFD3QN dynamically optimizes resource allocation and task offloading, enabling it to effectively balance energy consumption, minimize delays, and maintain high QoS satisfaction. These capabilities establish HMFD3QN as the most reliable and efficient algorithm, particularly in high-load situations.

It is worth noting that energy consumption and QoS task satisfaction are inherently conflicting objectives—achieving higher QoS satisfaction often requires selecting more powerful AI models, which increases energy consumption. Since this article prioritizes optimizing energy consumption, HMFD3QN demonstrates its effectiveness by striking a better balance between these competing objectives.

G. Performance Under Different Numbers of Terminals

This experiment explores the optimization capabilities of various algorithms in the lower layer by varying the number of terminals. As depicted in Fig. 11, significant differences in performance emerge among the five algorithms as the number of terminals increases (20, 40, and 60). HIQL and HMADDPG show poor scalability, resulting in elevated energy consumption and higher Lyapunov drift due to their inability to handle

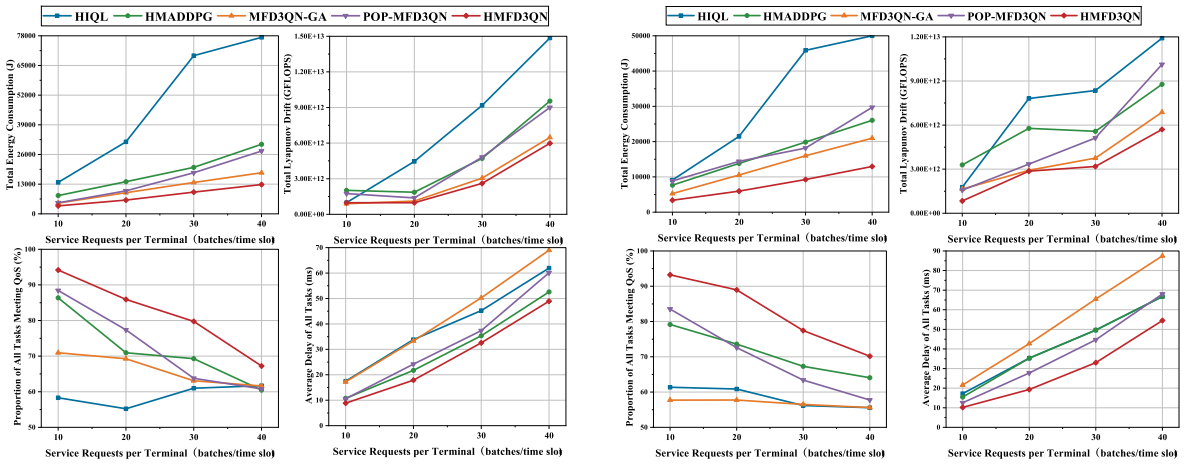


Fig. 10. Total energy consumption, Lyapunov drift, QoS task satisfaction ratio, and average task delay versus number of service requests per terminal (60 terminals, 8 computing nodes) (a) Altanta topology. (b) Ta2 topology.

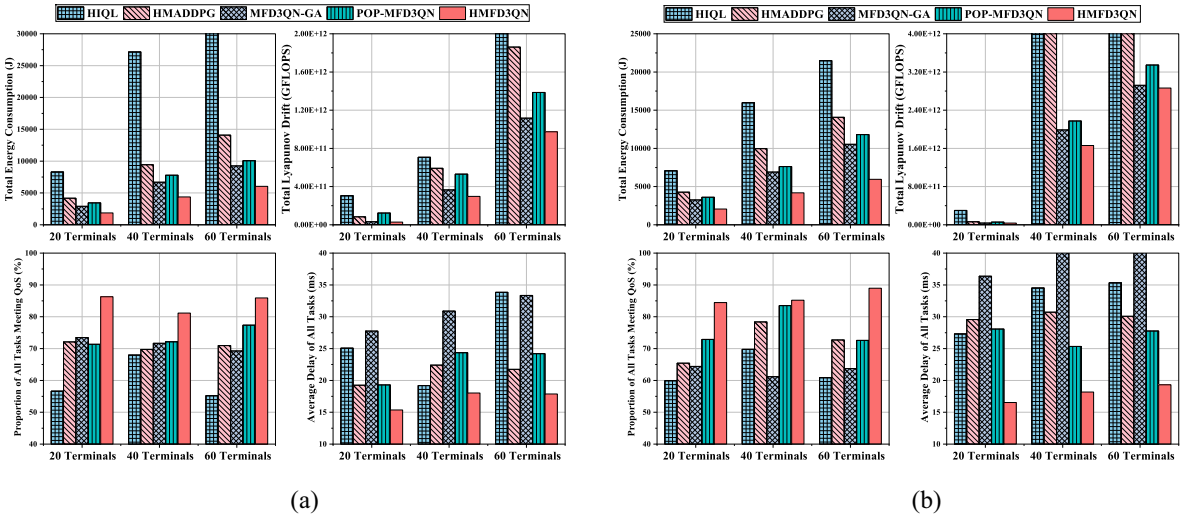


Fig. 11. Total Energy Consumption, Total Lyapunov Drift, QoS Task Satisfaction Ratio, and Average Task Delay versus Number of Terminals (8 Computing Nodes). (a) Altanta Topology. (b) Ta2 Topology.

the growing complexity of terminal interactions. MFD3QN-GA and POP-MFD3QN perform comparatively better, offering more stable energy consumption and lower Lyapunov drift. However, MFD3QN-GA suffers from the inherent limitations of its genetic algorithm, which struggles to optimize multiple objectives, leading to reduced QoS satisfaction and increased task delays. Similarly, the static service placement of POP-MFD3QN underperforms when compared to the dynamic placement strategy of HMFD3QN, as it lacks the flexibility to respond to real-time changes in network demands.

In stark contrast, HMFD3QN consistently delivers superior results, achieving the lowest energy consumption and Lyapunov drift while providing the highest QoS task satisfaction and shortest task delays. Specifically, HMFD3QN outperforms the second-best algorithm by reducing energy consumption by 37%, decreasing Lyapunov drift by 10%, improving QoS task satisfaction by 15%, and shortening average task delay by 22%. These improvements clearly demonstrate HMFD3QN's ability to effectively balance competing objectives and dynamically adapt to varying network conditions.

H. Performance Under Different Numbers of Computing Nodes

This experiment evaluates the adaptability of different algorithms in the upper layer by varying the number of computing nodes. As shown in Fig. 12, the performance of the five algorithms varies significantly, reflecting their ability to handle different scenarios. HIQL and HMADDPG exhibit limited scalability, resulting in inefficient optimization of energy consumption and Lyapunov drift, which hinders their overall performance. MFD3QN-GA and POP-MFD3QN achieve moderate success, with relatively low energy consumption and Lyapunov drift, earning them the second and third positions, respectively. However, these methods face limitations in

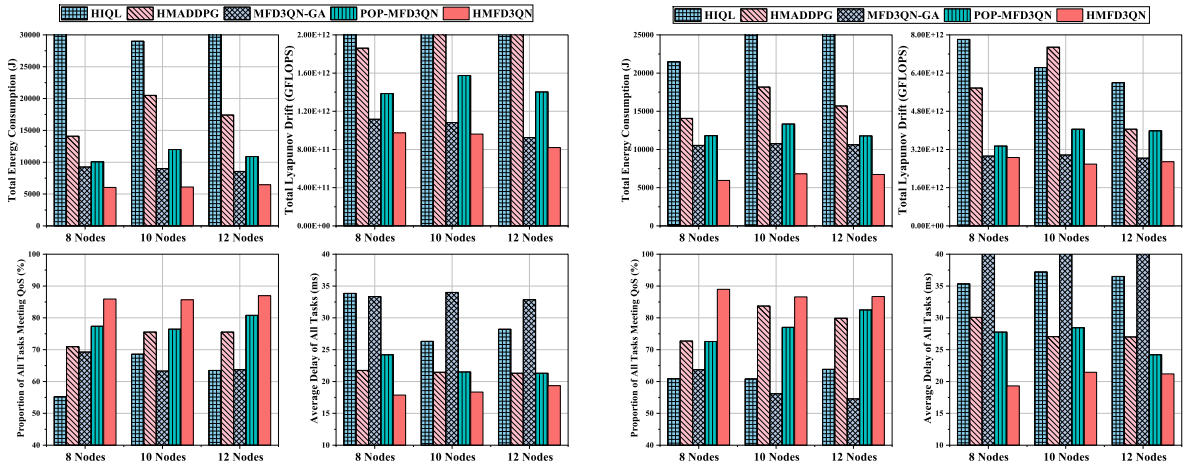


Fig. 12. Total Energy Consumption, Total Lyapunov Drift, QoS Task Satisfaction Ratio, and Average Task Delay versus Number of Computing Nodes (60 Terminals). (a) Altanta Topology. (b) Ta2 Topology.

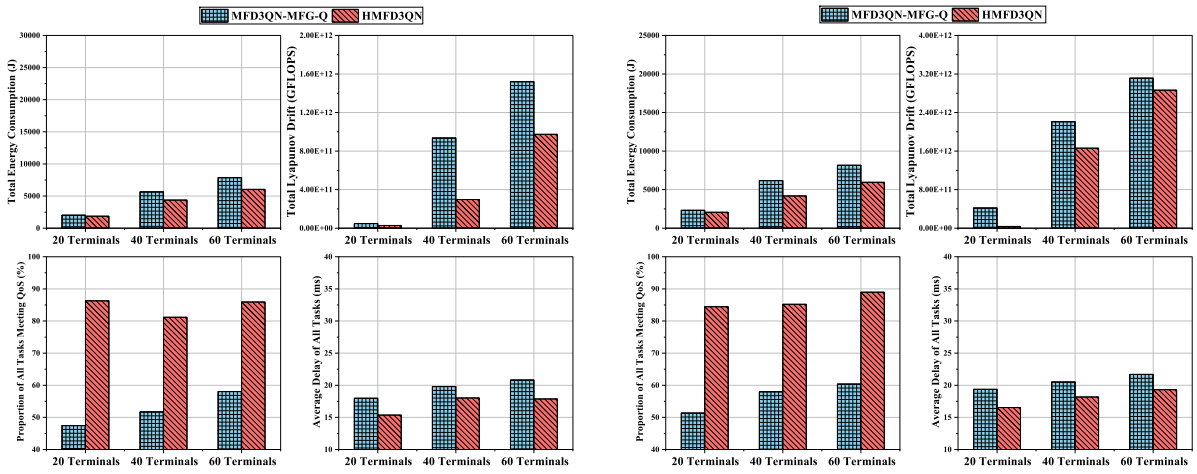


Fig. 13. Total Energy Consumption, Total Lyapunov Drift, QoS Task Satisfaction Ratio, and Average Task Delay versus Number of Terminals. (HMFD3QN versusMFD3QN-MFG-Q). (a) Altanta Topology. (b) Ta2 Topology.

multiobjective optimization and dynamic adaptability, which restrict their performance. In contrast, HMFD3QN emerges as the most effective algorithm across all metrics, providing a robust solution to the challenges posed by dynamic and complex network environments.

I. HMFD3QN versusMFD3QN-MFG-Q

MFG-*Q*, proposed in [35], is a mean field game guided deep reinforcement learning algorithm based on a single-agent DDPG framework, where the Actor's target network is replaced with mean field evolutionary dynamics to solve the mean field game. It addresses the task placement problem. In scenarios where tasks are homogeneous and specific QoS requirements are not considered, it simplifies the task placement problem into a task allocation ratio problem for servers, effectively transforming it into a single-agent decision problem. This approach reduces decision complexity, enhances efficiency, and minimizes task computation delays.

There is a fundamental difference in the use of mean field theory between MFG-*Q* and our proposed method. Specifically, MFG-*Q* employs mean field theory to address a specific problem: computing the Nash equilibrium of task allocation ratios among servers. In contrast, our method leverages mean field theory to model and resolve interactions among agents, thereby substantially reducing communication overhead between them. Furthermore, in our scenario, tasks represent diverse AI applications with specific QoS requirements (e.g., accuracy guarantees). Consequently, task placement decisions must consider both offloading locations and AI model selection, making the problem significantly more complex.

To evaluate its performance, we replace the lower-level MFD3QN (responsible for AI task placement decisions) in

HMFD3QN with MFG-Q, forming a variant called MFD3QN-MFG-Q. Fig. 13 compares the performance of the two algorithms across different terminal counts and network topologies. HMFD3QN consistently outperforms MFD3QN-MFG-Q, achieving average improvements of 20% in energy consumption, 45% in Lyapunov drift, 57% in QoS satisfaction, and 12% in task delay. The key difference lies in their design objectives. MFG-Q performs well in homogeneous task scenarios but struggles with diverse QoS demands, as shown by its 57% lower QoS task satisfaction ratio. In contrast, HMFD3QN uses a DMFG framework that integrates task-specific features (e.g., QoS requirements) into local agent observations, enabling agents to develop tailored policies and perform better in complex, large-scale environments.

VII. Conclusion

This article addresses the joint optimization of AI service placement, task scheduling, and computing resource allocation in the E-N-C system. Lyapunov optimization transforms the long-term energy minimization problem into short-term subproblems. To address the two-time-scale decision-making problem, the HMDP framework and the HMFD3QN algorithm are proposed. By integrating mean-field game theory, HMFD3QN reduces communication overhead and enhances scalability, enabling efficient optimization in large-scale multiagent scenarios. Furthermore, the computing resource allocation problem is proven to be convex, and a KKT conditions-based strategy is employed to efficiently solve it, simplifying the action space for reinforcement learning. Experimental results show that HMFD3QN outperforms baseline algorithms, achieving lower energy consumption, improved QoS satisfaction, queue stability, and scalability, especially in dynamic and large-scale environments. These results demonstrate HMFD3QN's robustness and effectiveness in addressing the complexities of 6G networks. Future work will focus on developing a real-world testbed to validate the proposed framework and algorithm in practical scenarios.

Appendix

Appendix: Proof

Based on inequality $([a-b]^+ + c)^2 \le a^2 + b^2 + c^2 + 2a(c-b)$, for each time slot, we have

$$\begin{aligned}
&L(Q(t+1)) - L(Q(t)) \\
&= \frac{1}{2}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\Big[\big(Q_{v,s}(t+1)\big)^2 - \big(Q_{v,s}(t)\big)^2\Big] \\
&= \frac{1}{2}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\Big\{\Big(\big[Q_{v,s}(t) - W_{v,s}(t)\big]^+ + A_{v,s}(t)\Big)^2 - Q_{v,s}^2(t)\Big\} \\
&\le \frac{1}{2}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\Big\{A_{v,s}^2(t) + W_{v,s}^2(t) + 2Q_{v,s}(t)\big[A_{v,s}(t) \\
&\quad - W_{v,s}(t)\big]\Big\} \\
&\le \frac{1}{2}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\Big\{A_{\max}^2(t) + W_{\max}^2(t) + 2Q_{v,s}(t)\big[A_{v,s}(t) \\
&\quad - W_{v,s}(t)\big]\Big\}. \qquad (54)
\end{aligned}$$

The upper bound of the drift-plus-penalty term can be calculated as

$$\begin{aligned}
&\Delta_T(Q(t)) + V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \\
&= \mathbb{E}\left[L(Q(t+T)) - L(Q(t)) + V\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \\
&= \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}(L(Q(\tau+1)) - L(Q(\tau))) + V\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \\
&\le \mathbb{E}\left[\frac{1}{2}\sum_{\tau=t}^{t+T-1}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}\Big(A_{\max}^2(\tau) + W_{\max}^2(\tau) + 2Q_{v,s}(\tau)(A_{v,s}(\tau) \right. \\
&\quad \left. - W_{v,s}(\tau)\Big) + V\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \\
&= \hat{B}T + \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}} Q_{v,s}(\tau)\big(A_{v,s}(\tau) - W_{v,s}(\tau)\big)\right. \\
&\quad \left. + V\sum_{\tau=t}^{t+T-1} E(\tau)|Q(t)\right] \qquad (55)
\end{aligned}$$

where $\hat{B} = (1/2)\sum_{v\in\mathcal{V}}\sum_{s\in\mathcal{S}}(A_{\max}^2(\tau) + W_{\max}^2(\tau))$.

References

[1] W. Fan et al., "DNN deployment, task offloading, and resource allocation for joint task inference in IIoT," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1634–1646, Feb. 2023.

[2] X. Xu, K. Yan, S. Han, B. Wang, X. Tao, and P. Zhang, "Learning-based edge-device collaborative DNN inference in IoVT networks," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7989–8004, Mar. 2024.

[3] X. Xie et al., "Memory-efficient and secure DNN inference on TrustZone-enabled consumer IoT devices," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2024, pp. 2009–2018.

[4] S. Yukun et al., "Computing power network: A survey," *China Commun.*, vol. 21, no. 9, pp. 109–145, 2024.

[5] Y. Yang et al., "Task-oriented 6G native-AI network architecture," *IEEE Netw.*, vol. 38, no. 1, pp. 219–227, Jan. 2024.

[6] A. Ali, R. Pinciroli, F. Yan, and E. Smirni, "Optimizing inference serving on serverless platforms," *Proc. VLDB Endowment*, vol. 15, no. 10, pp. 2071–2084, 2022.

[7] Y. Yang et al., "INFless: A native serverless system for low-latency, high-throughput inference," in *Proc. 27th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2022, pp. 768–781.

[8] Z. Hong et al., "Optimus: Warming serverless ML inference via inter-function model transformation," in *Proc. 19th Eur. Conf. Comput. Syst.*, 2024, pp. 1039–1053.

[9] Y. Chen, X. Hu, S. Gong, Z. Su, and B. Gu, "Multitime scale service caching and pricing in MEC systems with dynamic program popularity," *IEEE Internet Things J.*, vol. 12, no. 11, pp. 15438–15452, Jun. 2025.

[10] M. R. Abedi, N. Mokari, M. R. Javan, H. Saeedi, E. A. Jorswieck, and N. Zorba, "Low complexity and mobility-aware robust radio, storage, computing, and cost management for cellular vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 3327–3344, Feb. 2025.

[11] Z. Liang, Y. Liu, T.-M. Lok, and K. Huang, "A two-timescale approach to mobility management for multicell mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10981–10995, Dec. 2022.

[12] B. Xu et al., "Mobility-aware task offloading in industrial fog networks: A Submodular-based MARL approach," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 10795–10807, Apr. 2025.

[13] R. Men, X. Fan, K.-L. A. Yau, A. Shan, and G. Yuan, "Hierarchical aerial computing for task offloading and resource allocation in 6G-enabled vehicular networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 4, pp. 3891–3904, Jul./Aug. 2024.

[14] Y. Wu, J. Wu, L. Chen, B. Liu, M. Yao, and S. K. Lam, "Share-aware joint model deployment and task offloading for multi-task inference," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 6, pp. 5674–5687, Jun. 2024.

[15] L. Wang, X. Ren, C. Zhao, F. Zhao, and S. Yang, "MPDM: A multi-paradigm deployment model for large-scale edge-cloud intelligence," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8773–8785, May 2023.
[16] L. Huang, Y. Wu, J. M. Parra-Ullauri, R. Nejabati, and D. Simeonidou, "AI model placement for 6G networks under epistemic uncertainty estimation," in *Proc. ICC IEEE Int. Conf. Commun.*, 2024, pp. 5521–5526.
[17] W. Zhang, S. Zeadally, W. Li, H. Zhang, J. Hou, and V. C. Leung, "Edge AI as a service: Configurable model deployment and delay-energy optimization with result quality constraints," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1954–1969, Apr.–Jun. 2023.
[18] J. Li, F. Lin, L. Yang, and D. Huang, "AI service placement for multi-access edge intelligence systems in 6G," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1405–1416, May/Jun. 2023.
[19] Y. Lu, M. Zhang, and M. Tang, "Caching for edge inference at scale: A mean field multi-agent reinforcement learning approach," in *Proc. IEEE GLOBECOM Global Commun. Conf.*, 2023, pp. 332–337.
[20] T. Chen, Q. Tang, and G. Liu, "Efficient task scheduling and resource allocation for AI training services in native AI wireless networks," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, 2023, pp. 637–642.
[21] Z. Zeng, J. Huang, J. Wu, and J. Wu, "Joint request offloading and resource allocation for energy efficient D2D enabled multi-type inference services," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Social Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, 2023, pp. 213–220.
[22] Y. Yang, R. Luo, M. Li, M. Zhou, W. Zhang, and J. Wang, "Mean field multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5571–5580.
[23] P.-G. Ye, Y.-G. Wang, and W. Tang, "S-MFRL: Spiking mean field reinforcement learning for dynamic resource allocation of D2D networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1032–1047, Jan. 2023.
[24] Y. Emami, H. Gao, K. Li, L. Almeida, E. Tovar, and Z. Han, "Age of information minimization using multi-agent UAVs based on AI-enhanced mean field resource allocation," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 13368–13380, Sep. 2024.
[25] C. Ji, A. Gao, S. Liu, and W. Duan, "Mean-field multi-agent reinforcement learning for UAV assisted secure data dissemination," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2023, pp. 908–913.
[26] Z. Zhou, L. Qian, and H. Xu, "Decentralized multi-agent reinforcement learning for large-scale mobile wireless sensor network control using mean field games," in *Proc. 33rd Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2024, pp. 1–6.
[27] J. Ren et al., "MeFi: Mean field reinforcement learning for cooperative routing in wireless sensor network," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 995–1011, Jan. 2024.
[28] J. Chen, X. Xiong, X. Guan, Q. Zhang, M. Xia, and J. Zhao, "Multi-agent reinforcement learning connectivity optimization algorithm in vehicular networks," in *Proc. IEEE/CIC Int. Conf. Commun. China (ICCC Workshops)*, 2024, pp. 364–368.
[29] M. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. San Rafael, CA, USA: Morgan & Claypool Publishers, 2010.
[30] R. Huang, M. Guo, C. Gu, S. He, J. Chen, and M. Sun, "Toward scalable and efficient hierarchical deep reinforcement learning for 5G RAN slicing," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 4, pp. 2153–2162, Dec. 2023.
[31] X. Li, Y. Zhang, H. Ding, and Y. Fang, "Intelligent spectrum sensing and access with partial observation based on hierarchical multi-agent deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 23, no. 4, pp. 3131–3145, Apr. 2024.
[32] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, nos. 1–2, pp. 181–211, 1999.
[33] H. E. Stanley, *Phase Transitions and Critical Phenomena*. Oxford, U.K.: Clarendon Press, 1971.
[34] H. Gao et al., "Energy-efficient velocity control for massive numbers of UAVs: A mean field game approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 6, pp. 6266–6278, Jun. 2022.
[35] D. Shi, H. Gao, L. Wang, M. Pan, Z. Han, and H. V. Poor, "Mean field game guided deep reinforcement learning for task placement in cooperative multiaccess edge computing," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9330–9340, Oct. 2020.
[36] S. G. Subramanian, M. E. Taylor, M. Crowley, and P. Poupart, "Decentralized mean field games," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 9439–9447.
[37] "Study on management aspects of network digital twin (Release 19)," 3GPP, Sophia Antipolis, France, Rep. TR 28.915, Feb. 2024.
[38] "Network digital twin," ETSI, Sophia Antipolis, France, Rep. GR ZSM 015, Feb. 2024.
[39] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[40] S. Orlowski, M. Pióro, A. Tomaszewski, and R. Wessäly, "SNDlib 1.0—Survivable network design library," *Networks*, vol. 55, no. 3, pp. 276–286, 2010. [Online]. Available: http://www3.interscience.wiley.com/journal/122653325/abstract
[41] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6382–6393.
[42] J.-Y. Audibert, R. Munos, and C. Szepesvári, "Exploration–exploitation tradeoff using variance estimates in multi-armed bandits," *Theor. Comput. Sci.*, vol. 410, no. 19, pp. 1876–1902, 2009.

Zhenyu Zhang is currently pursuing the Ph.D. degree with Beijing University of Posts and Telecommunications, Beijing, China.

His research interests include causal inference and wireless network resource allocation.

Lu Lu received the master's degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2005.

She is the Deputy Director of the Department of Basic Network Technology of CMRI, the Leader of Core Network Group of CCSA TC5, and the Vice-Chairman of ITU-T SG13. Her main research direction covers 5G/6G architecture and computing power network.

Qin Li received the master's degree from Chongqing University, Chongqing, China, in 2002.

She is a Senior Researcher of China Mobile Research Institute and has engaged in technical and application research on core network and content network for more than 20 years. Her research interests cover 5G/6G architecture, network intelligence, and digital twin network.

Yuhao Chai is currently pursuing the Ph.D. degree with Beijing University of Posts and Telecommunications, Beijing, China.

His research interests include game theory, 6G networks, and resource allocation.

Di Wu received the B.E. degree in communication engineering from Yanshan University, Qinhuangdao, China. She is currently pursuing the M.S. degree with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China.

Her research interests include 6G deterministic networks, resource allocation, and reinforcement learning.

Yong Zhang (Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications (BUPT), Beijing, China in 2007.

He is a Professor with the School of Electronic Engineering, BUPT. He is currently the Director of Fab.X Artificial Intelligence Research Center, BUPT. He is the Deputy Head of the Mobile Internet Service and Platform Working Group, China Communications Standards Association. He has authored or co-authored more than 80 papers and holds 30 granted China patents. His research interests include Artificial Intelligence, wireless communication, and Internet of Things.